TRAILLINK: HYBRID COMMUNICATION & SAFETY SYSTEM

INDIVIDUAL THESIS

Nethula Dulsara

(IT22544490)

Dissertation submitted in partial fulfilment of the requirements for the Bachelor of Science (Hons) in Information Technology Specializing in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

October 2026

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy. I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Student name: Dulsara H G N

Student ID: IT22544490

Signature: _____ Date: _____11.10.2026_____

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Role and name	Signature	Date
Supervisor: Mr.Vishan Jayasinghearachchi		
Co-supervisor: Ms. Sasini Hathurusinghe		

ACKNOWLEDGEMENTS

I wish to express my deepest gratitude to my academic supervisor, Ms. Sasini Hathurusinghe, and my co-supervisor, Mr. Vishan Jayasinghearachchi, from the Department of Information Technology, Faculty of Computing, Sri Lanka Institute of Information Technology (SLIIT). Their insightful academic critique, technical guidance, and continuous encouragement throughout the conception, architectural design, and empirical evaluation of this research have been invaluable.

I would also like to extend my sincere appreciation to the academic and technical staff of the Faculty of Computing for providing the laboratory facilities and academic environment essential for completing this study. Special thanks are conveyed to my fellow TrailLink project colleagues—responsible for the adjacent messaging, voice radio, and link-quality modules—whose cohesive collaboration enabled seamless inter-component integration and field testing.

Finally, I express my heartfelt gratitude to my family and friends for their enduring patience, understanding, and moral support throughout the tenure of my undergraduate degree.

ABSTRACT

Reliable spatial orientation and rapid crisis dissemination represent indispensable operational requirements when wilderness expeditions, backcountry search-and-rescue teams, and disaster response personnel operate beyond terrestrial telecommunications coverage. Conventional mobile navigation and messaging applications are fundamentally predicated on persistent cloud access, causing them to degrade abruptly into inert interfaces when centralized cellular infrastructure is damaged, saturated, or absent. While Mobile Ad-Hoc Network (MANET) paradigms provide theoretical routing models, deploying these protocols on commercial off-the-shelf Android smartphones is constrained by operating-system sandboxing that prohibits raw Layer-3 packet injection without invasive root privileges.

This research investigates the design, implementation, and empirical validation of two foundational subsystems within the TrailLink mobile platform: (1) a local-first spatial awareness and vector/raster mapping pipeline, and (2) a bounded, priority-driven distress alert dissemination protocol (Emergency SOS) featuring transactional cloud state reconciliation. Executing entirely in unprivileged Android user-space, the system couples Bluetooth Low Energy (BLE) neighbor discovery with dynamic Wi-Fi Direct socket promotion via the Google Nearby Connections framework. A local relational SQLite engine synchronously commits spatial fixes and distress events, enforcing transactional rollback guarantees under sudden power exhaustion. Real-time mapping integrates pre-rendered offline MBTiles archives with a dynamic three-tier temporal freshness function to prevent reliance on stale coordinates. For crisis coordination, an out-of-order priority queue expedites SOS payloads over standard media traffic, bounded by a time-to-live ($TTL = 8$) hop constraint and an idempotent duplicate-filtering ledger to avert broadcast storms. Re-established backhaul links trigger an automated, batched reconciliation engine that commits accumulated field records to remote MongoDB instances.

Empirical field trials executed across physical Android handsets across open grounds, light timber, and concrete-partitioned structures demonstrate a 99.5% line-of-sight packet delivery ratio at 20 meters, with SOS dispatch latencies consistently below 850 ms through multi-wall obstructions. The system proves that life-critical spatial coordination and distress signaling can be achieved reliably on unmodified consumer smartphones in zero-infrastructure environments.

Keywords: Offline-First Architectures, Mobile Ad-Hoc Networks, Emergency SOS, Geospatial Synchronization, Wi-Fi Direct.

TABLE OF CONTENTS

1 INTRODUCTION....1
1.1 Overall project context....1
1.2 Individual component context....2
1.3 Problem statement....3
1.4 Research gap....4
1.5 Aim and objectives....5
1.6 Research questions or hypotheses....6
1.7 Scope limitations and assumptions....7
1.8 Individual contribution....8
1.9 Thesis structure....9
2 LITERATURE REVIEW AND RELATED WORK....10
2.1 Review method and scope....10
2.2 Domain and theoretical foundations....11
2.3 Related research....14
2.4 Existing systems products and techniques....16
2.5 Gap synthesis and conceptual framework....18
3 RESEARCH METHODOLOGY....20
3.1 Research design....20
3.2 Data participants or study materials....21
3.3 Experimental or study procedure....22
3.4 Measures and evaluation metrics....24
3.5 Data analysis....25
3.6 Ethics privacy security and responsible AI....26
3.7 Reproducibility and research artefacts....27
4 PRODUCT AND SOLUTION DESIGN....28
4.1 Requirements....28
4.2 Component architecture within the complete solution....30
4.3 Design alternatives and decisions....32
4.4 Detailed design....34
4.5 Standards and professional practice....38
5 IMPLEMENTATION INTEGRATION AND TESTING....40
5.1 Development environment and technology stack....40
5.2 Component implementation....41
5.3 Integration with the other components....45
5.4 Verification and software testing....46
5.5 Deployment and operation....48
6 RESULTS AND VALIDATION....50
6.1 Research results....50

6.2 Product and technical results 53
6.3 User or stakeholder validation 55
6.4 Integration results 55
6.5 Summary of findings 56
7 DISCUSSION 57
7.1 Interpretation of findings 57
7.2 Comparison with literature baselines and existing solutions 58
7.3 Research contribution 59
7.4 Product contribution 60
7.5 Limitations and threats to validity 60
7.6 Implications 61
8 COMMERCIALISATION AND SUSTAINABILITY 62
8.1 Value proposition and target users 62
8.2 Market and competitive position 63
8.3 Business or sustainability model 64
8.4 Scalability intellectual property and compliance 65
8.5 Contribution to the Sustainable Development Goals 66
9 CONCLUSION AND FUTURE DIRECTIONS 67
9.1 Conclusion 67
9.2 Achievement of objectives 68
9.3 Individual research and product contributions 69
9.4 Future directions 70
REFERENCES 71
GLOSSARY 74
APPENDICES 76
Appendix A Research instruments or extended results 76
Appendix B Academic integrity and AI use disclosure 77

LIST OF FIGURES

- Figure 2.1 Conceptual Framework for TrailLink's Offline Spatial Awareness and Emergency SOS Subsystem
  Source: Original in Color
- Figure 3.1 Design Science Research Methodology (DSRM) Workflow for TrailLink
  Source: Original in Color
- Figure 3.2 Spatial Topologies and Physical Obstructions Across Testing Environments
  Source: Original in Color
- Figure 4.1 TrailLink Global Multi-Modal Architecture Highlighting the Individual Subsystem
  Source: Original in Color
- Figure 4.2 Exhaustive Internal Subsystem Architecture of the Location, Mapping, and SOS Component
  Source: Original in Color
- Figure 4.3 End-to-End Sequence Diagram of Emergency SOS Dispatch, Relaying, and Cloud Sync
  Source: Original in Color
- Figure 5.1 Multi-Modal Integration Points Across the Four Subsystems
  Source: Original in Color
- Figure 6.1 Packet Delivery Ratio (PDR) vs. Physical Separation Distance Across Terrains
  Source: Original in Color
- Figure 6.2 End-to-End SOS Latency Across Separation Distances Across Terrains
  Source: Original in Color

LIST OF TABLES

- Table 1.1 Comparison of Existing Paradigms and the TrailLink Research Gap
- Table 2.1 Analytical Feature Comparison Across Mobile Offline Platforms
- Table 3.1 Hardware and Environmental Specifications of Experimental Handsets
- Table 4.1 MoSCoW Prioritization of System Requirements
- Table 5.1 Software Libraries and Framework Dependencies
- Table 6.1 Empirical Packet Delivery Ratio (PDR) Across Terrains (n = 50 trials per condition)
- Table 6.2 End-to-End Transmission Latency Across Environments (n = 50 trials per condition)
- Table 6.3 Subsystem Current Draw, Power Dissipation, and Projected Battery Longevity
- Table 6.4 Summary of Hypothesis Testing Outcomes
- Table 8.1 Competitive Positioning Matrix
- Table 8.2 Alignment with United Nations Sustainable Development Goals (SDGs)

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AAC	Advanced Audio Coding
AODV	Ad hoc On-Demand Distance Vector
API	Application Programming Interface
BLE	Bluetooth Low Energy
DTN	Delay-Tolerant Networking
GNSS	Global Navigation Satellite System
GPS	Global Positioning System
IEEE	Institute of Electrical and Electronics Engineers
IP	Internet Protocol
JSON	JavaScript Object Notation

MAC	Media Access Control
MANET	Mobile Ad-Hoc Network
MTU	Maximum Transmission Unit
OS	Operating System
OSM	OpenStreetMap
P2P	Peer-to-Peer
PCM	Pulse-Code Modulation
PDR	Packet Delivery Ratio
REST	Representational State Transfer
RSSI	Received Signal Strength Indicator
RTT	Round-Trip Time
SAR	Search and Rescue
SDK	Software Development Kit
SOS	Save Our Souls (Emergency Distress Signal)
TTL	Time-To-Live
UUID	Universally Unique Identifier
WAL	Write-Ahead Logging
Wi-Fi	Wireless Fidelity

CHAPTER 1: INTRODUCTION

1.1 Overall project context

Terrestrial telecommunications networks—encompassing cellular base transceiver stations (BTS), fiber backhaul rings, and commercial satellite uplinks—constitute the backbone of modern digital communication. However, these systems exhibit acute architectural fragility during natural disasters, geophysical crises, and deep wilderness expeditions. Cyclones, floods, seismic events, and severe weather patterns frequently trigger structural collapses of antenna masts, prolonged grid power outages, and catastrophic physical cable disruptions. In remote recreational environments such as highland nature reserves, deep gorges, and dense forest canopies, commercial cellular coverage is non-existent due to the economic infeasibility of deploying rural telecommunications infrastructure.

When teams traverse these disconnected environments—whether they are volunteer search-and-rescue (SAR) personnel, park rangers, geological surveyors, or wilderness hikers—the lack of communication infrastructure introduces severe operational vulnerabilities. Commercial consumer applications (e.g., WhatsApp, Signal, Google Maps) rely strictly on continuous client-server handshakes. When backhaul transit collapses, these applications fail completely, locking users out of collaborative coordination, spatial tracking, and distress notification.

To resolve this technological void, the collaborative research initiative TrailLink was conceived. TrailLink is an infrastructure-independent, peer-to-peer mobile mesh collaboration ecosystem engineered specifically for commercial off-the-shelf (COTS) Android devices. Without requiring external hardware accessories or operating-system modifications, TrailLink establishes a localized multi-modal communication mesh across proximate handsets. The broader TrailLink platform integrates four synchronized services: 1. Reliable Store-and-Forward Text Messaging: Asynchronous transactional chatter across intermittent links. 2. Dual-Pipeline Push-to-Talk (PTT) Audio: Short-burst compressed voice memos alongside low-latency live PCM-16 voice streaming with jitter buffering. 3. Dynamic Link Quality & Mesh Assessment: Real-time link health classification ($Q \in [0,100]$)

driving topology adaptations. 4. Geospatial Awareness & Bounded Emergency SOS: Real-time location tracking over offline maps, out-of-order distress coordination, and automated upstream cloud state reconciliation.

1.2 Individual component context

Within the multi-faceted TrailLink ecosystem, this thesis focuses on the most safety-critical subsystem: Offline-First Location Sharing, Cached Geospatial Mapping, and Priority-Based Bounded Emergency Coordination (SOS) with Cloud Data Reconciliation.

While conversational text and voice channels serve day-to-day organizational needs, geospatial tracking and distress propagation constitute life-safety capabilities. If a team member becomes disoriented, sustains physical trauma, or encounters hazardous terrain, situational awareness requires teammates to pinpoint their exact geographical coordinates in real time.

This individual component is responsible for four key functions: - Local-First Coordinate Capture & Modeling: Continuously acquiring high-accuracy spatial coordinates from the device's internal GNSS receiver via unprivileged user-space APIs and enforcing strict spatio-temporal serialization. - Decentralized Geospatial Gossiping & Offline Rendering: Packaging coordinates into compact binary-compatible JSON envelopes, broadcasting them across proximate peer nodes over ad-hoc Wi-Fi Direct links, and visualizing teammates' locations over pre-cached vector/raster map archives without an active internet connection. - Bounded Out-of-Band Emergency SOS Coordination: Providing an emergency trigger mechanism that generates high-priority distress alerts. These packets bypass standard routine communication buffers, propagate through a multi-hop mesh bounded by strict Time-To-Live (TTL) controls, suppress broadcast storms using a local deduplication ledger, trigger localized audio-visual alarms, and elicit explicit peer acknowledgments. - Transactional Dual-State Cloud Reconciliation: Implementing an automated cloud bridge (BridgeEngine) that detects the return of internet connectivity on any node, batch-synchronizes accumulated offline location histories

and emergency incident records to a remote MongoDB database, and ensures consistency via transactional rollbacks.

1.3 Problem statement

Modern wilderness navigation and team safety suffer from a fundamental disconnect: commercial mobile operating systems enforce rigid security boundaries that obstruct ad-hoc networking, while consumer safety software assumes ubiquitous cloud connectivity.

Specifically, the core problem decomposes into three distinct challenges: 1. Operating System Privilege Boundaries: Classic mobile ad-hoc routing protocols (such as AODV and OLSR) operate at Network Layer 3. Stock Android security models prevent unprivileged applications from manipulating kernel routing tables, setting raw network interfaces into ad-hoc mode, or broadcasting raw 802.11 frames. Existing academic solutions bypass this by requiring root access, which invalidates hardware warranties, introduces severe security vulnerabilities, and prevents broad public adoption. 2. Bandwidth and Resource Contention During Emergencies: In an ad-hoc wireless mesh operating over unlicensed 2.4 GHz and 5 GHz spectrum, wireless bandwidth is scarce. If an emergency distress signal is injected into a peer network that is concurrently relaying bulky voice streams or store-and-forward text logs, traditional First-In, First-Out (FIFO) queueing introduces critical packet latency or packet drops. Furthermore, unrestrained multi-hop broadcasting can induce catastrophic broadcast storms, saturating the wireless medium with duplicate packets and accelerating battery depletion. 3. Dual-State Data Inconsistency in Offline-First Systems: Mobile nodes operating in disconnected wilderness zones accumulate large volumes of geospatial coordinates and emergency state changes. Standard cloud-centric database patterns fail because they treat local devices as ephemeral caches. Without robust local-first relational modeling, sudden application crashes, background OS kills, or battery exhaustion can cause catastrophic loss of audit logs. Conversely, naive synchronization upon internet re-establishment risks duplicate record creation, server overload, and race conditions.

1.4 Research gap

A comprehensive survey of current literature and existing industry platforms highlights significant functional gaps:

- Commercial Satellite and Dedicated Mesh Appliances: Proprietary transceivers like Garmin inReach and goTenna Pro provide long-range off-grid communications. However, they require users to purchase expensive proprietary hardware, maintain costly monthly subscriptions, and carry extra peripheral bulk. When batteries drain or hardware is damaged, the user is left with no alternative.
- Academic Layer-3 MANET Implementations: Systems such as the Serval Project demonstrated the viability of mobile mesh communications, but relied on custom kernel patches and root privileges, making them unusable on consumer devices.
- Application-Layer Mobile Mesh Messengers: Platforms such as Briar and Bridgefy enable application-layer multi-hop messaging over Bluetooth and Wi-Fi. However, Briar focuses strictly on delayed store-and-forward text over Tor, lacking real-time spatial visualization, live mapping, and emergency signaling. Bridgefy focuses exclusively on short text payloads without synchronized offline spatial mapping, topological decay tracking, or transactional database bridging to enterprise servers.
- Absence of Unified Spatial Freshness & Priority Routing: Existing software does not integrate offline vector map rendering, spatio-temporal freshness decay metrics (Δt), out-of-order priority emergency scheduling, and atomic cloud database reconciliation into a single user-space application framework.

Table 1.1 summarizes the research and engineering gap addressed by this research.

Table 1.1: Comparison of Existing Paradigms and the TrailLink Research Gap

System / Paradigm	Unmodified Stock Handset	Zero Extra Hardware	Offline Geospatial Map	Prioritized Emergency SOS	Automated Cloud Bridge	Broadcast Storm Protection
Garmin inReach	No	No	Limited	Yes	Proprietary	N/A
goTenna Pro	Yes	No	Partial	Yes	No	N/A
Serval	No	Yes	No	No	No	Partial

Mesh	(Requires Root)					
Briar Project	Yes	Yes	No	No	No	Yes
Bridgefy SDK	Yes	Yes	No	No	No	No
TrailLink (This Work)	**Yes**	**Yes**	**Yes**	**Yes**	**Yes**	**Yes**

1.5 Aim and objectives

1.5.1 Project aim

The primary aim of this research is to design, implement, and empirically validate an infrastructure-independent, unprivileged, user-space location-sharing and emergency-coordination subsystem for consumer Android smartphones. This subsystem ensures continuous spatial situational awareness, reliable bounded distress dispatch, and consistent cloud database reconciliation across zero-infrastructure and disaster environments.

1.5.2 Specific research and technical objectives

To achieve this aim, the following concrete technical objectives were established: 1. Design a User-Space Ad-Hoc Transport and Spatial Ingestion Architecture: Formulate an unprivileged peer-to-peer transport architecture utilizing Google Nearby Connections (BLE discovery promoted to Wi-Fi Direct) to acquire, serialize, and broadcast local GNSS coordinates within a 512-byte MTU-compatible JSON framing structure without requiring Android root privileges. 2. Develop an Offline Geospatial Engine with Dynamic Temporal Freshness: Implement an on-device mapping subsystem utilizing embedded MBTiles vector archives and cached OpenStreetMap raster tiles, incorporating a mathematical temporal decay function:

$$\mathcal{F}_{\text{freshness}}(\Delta t)$$

to classify coordinate currency (Fresh, Stale, Expired) and render real-time situational badges. 3. Formulate a Priority-Driven Bounded Emergency SOS Protocol: Construct an out-of-order dispatch mechanism that intercepts emergency events, bypasses background data buffers, governs multi-hop propagation via strict Time-To-Live ($TTL = 8$) and hop boundaries ($hopCount \leq 5$), suppresses broadcast storms via an on-device deduplication ledger (processed_offline_packets), and enforces bidirectional end-to-end delivery acknowledgments (sos_ack). 4. Engineer an Atomic Dual-State SQLite-to-MongoDB Reconciliation Engine: Develop a resilient local relational database schema across location_updates, teammate_locations, and emergency_events that synchronously logs events to disk. Couple this schema with an automated BridgeEngine that batches unsynced records upstream to remote MongoDB endpoints upon internet restoration with transactional rollback guarantees. 5. Empirically Validate Delivery and Latency Across Physical Environments: Conduct field trials across two commercial Google Pixel 7 smartphones in three distinct real-world propagation environments (Open Area, Light Forest, and Between Classrooms) across separation distances from 10 m to 50 m to measure Packet Delivery Ratio (PDR), end-to-end latency, and database batch-synchronization efficiency.

1.6 Research questions or hypotheses

1.6.1 Research questions

- RQ1: Can an unprivileged, application-layer transport protocol running on unmodified consumer Android handsets achieve sufficient packet delivery rates and low latencies for emergency distress dispatches across varied physical terrains without infrastructure?
- RQ2: How effectively can an out-of-order priority queuing and deduplication ledger suppress broadcast loops while ensuring emergency packets propagate through ad-hoc hops ahead of routine conversational text and voice traffic?
- RQ3: How can local-first relational data structures guarantee zero data loss during sudden mobile operating-system termination while maintaining non-blocking transactional consistency when reconciling offline logs with remote cloud databases?

1.6.2 Research hypotheses

- Hypothesis 1 (H_1): By utilizing hybrid BLE discovery promoted to high-bandwidth Wi-Fi Direct communication endpoints, an unprivileged user-space protocol can maintain a Packet Delivery Ratio (PDR) exceeding 90% at separation distances up to 30 meters in unobstructed and lightly obstructed outdoor environments.
- Hypothesis 2 (H_2): Bounded priority-queued distress signaling will achieve an average end-to-end delivery latency of less than 1.5 seconds under physical line-of-sight and multi-wall indoor obstructions, satisfying the internationally recognized human-response threshold for emergency distress alarms.
- Hypothesis 3 (H_3): Synchronous local SQLite transactional logging combined with batched HTTP-REST state reconciliation will successfully commit accumulated offline records (up to 100 queued items) to a cloud database upon link restoration without generating duplicate entries, data loss, or user-interface freezing.

1.7 Scope limitations and assumptions

1.7.1 Scope limitations

- Radio Hardware Diversity: Empirical evaluations are conducted using identical Google Pixel 7 hardware. Radio sensitivity, antenna geometry, and BLE driver behavior may vary across diverse third-party Android OEMs.
- Atmospheric Conditions: Experimental field trials were completed under fair, dry weather conditions. Severe atmospheric precipitation (heavy rainstorms, dense wet foliage) may induce additional attenuation along 2.4 GHz and 5 GHz frequency paths.
- Operating System Constraints: The system operates within Android user-space and is subject to OEM-specific background battery optimization policies, requiring explicit foreground-service configurations to ensure persistent operation.
- Scale of Empirical Testbed: Physical verification is bounded by a two-handset point-to-point testbed, with multi-hop behavior validated programmatically through routing logic assertions.

1.7.2 Assumptions

- Participating mobile handsets possess functional hardware radios: a GNSS receiver, Bluetooth Low Energy controller (v4.2 or higher), and a Wi-Fi chipset supporting Wi-Fi Direct.
- Users operating in off-grid environments pre-download the relevant geographic MBTiles map packages prior to entering zones devoid of network infrastructure.

- At least one member device will eventually encounter internet connectivity (acting as a "data mule" or gateway bridge) to reconcile accumulated expedition records with the central cloud database.

1.8 Individual contribution

While TrailLink was developed as a collaborative group research effort, the author independently conceived, designed, implemented, and empirically validated the following contributions:

1. Architected the Local-First Geospatial Pipeline: Designed the data-flow architecture that samples GNSS coordinates, translates them into compact serializable JSON payloads (under 512 bytes), and updates the local spatial database.
2. Implemented the Offline Map Rendering Engine: Built the map visualization system within Flutter using offline MBTiles vector and raster caches. Formulated the temporal freshness decay logic that dynamically categorizes teammate coordinate markers into Fresh, Old, and Stale states based on elapsed time (Δt).
3. Engineered the Bounded Emergency SOS Protocol: Authored the out-of-order priority queueing system, the bounded multi-hop forwarding routine ($\text{TTL} = 8$, $\text{hopCount} \leq 5$), the split-horizon broadcast suppression logic, and the deduplication verification algorithm based on processed_offline_packets.
4. Constructed the Relational Database & Cloud Reconciliation Engine (BridgeEngine): Developed the SQLite schema (location_updates, teammate_locations, emergency_events), established indexed transactions to prevent database lockups, and authored the synchronization engine that reconciles field records with MongoDB over REST endpoints.
5. Conducted Empirical Field Benchmarking: Designed and executed the empirical test protocol using two Google Pixel 7 smartphones, recording 50 test iterations per scenario across Open Area, Light Forest, and Between Classrooms configurations to evaluate PDR, latency, and synchronization overhead.

1.9 Thesis structure

The remainder of this dissertation is organized as follows: - Chapter 2: Literature Review and Related Work: Examines theoretical foundations of MANETs and DTNs, surveys commercial and academic mobile ad-hoc solutions, analyzes local-first database sync paradigms, and establishes the conceptual framework. - Chapter 3: Research Methodology: Outlines the design-science and experimental research

methodology, field setup, measurement variables, evaluation metrics, and ethical considerations. - Chapter 4: Product and Solution Design: Details the comprehensive software architecture, component requirements, data models, algorithm design, sequence interactions, and engineering standards. - Chapter 5: Implementation, Integration, and Testing: Documents the development environment, Flutter and native Android implementations, inter-subsystem integration, unit testing, and deployment configurations. - Chapter 6: Results and Validation: Presents empirical field results, statistical analyses of Packet Delivery Ratios, transmission latencies across varying terrains, and cloud reconciliation benchmarks. - Chapter 7: Discussion: Critically interprets empirical findings, benchmarks results against literature baselines, analyzes research and product contributions, evaluates threats to validity, and details practical implications. - Chapter 8: Commercialisation and Sustainability: Outlines the commercial value proposition, target market segments, competitive positioning, business sustainability models, scalability considerations, and alignment with the United Nations Sustainable Development Goals (SDGs). - Chapter 9: Conclusion and Future Directions: Summarizes primary findings, reviews the achievement of research objectives, reiterates individual contributions, and identifies pathways for future technical development. - References, Glossary, and Appendices: Provides IEEE-formatted citations, domain terminology definitions, raw testing data logs, and the academic integrity AI disclosure.

CHAPTER 2: LITERATURE REVIEW AND RELATED WORK

2.1 Review Method and Scope

To establish a rigorous theoretical foundation and systematically map the research landscape surrounding infrastructure-independent communications, this review adopts the Preferred Reporting Items for Systematic Reviews and Meta-Analyses (PRISMA) framework. The investigation focuses on mobile ad-hoc networking, delay-tolerant data dissemination, local-first data persistence, and emergency distress protocols operable on resource-constrained consumer mobile hardware.

2.1.1 Search Strategy and Database Selection

A comprehensive multi-database search was conducted across leading scientific repositories, including: - IEEE Xplore Digital Library - ACM Digital Library - ScienceDirect (Elsevier) - SpringerLink - USENIX Association Proceedings

The search string combined targeted keywords utilizing Boolean operators: ("mobile ad-hoc network" OR "MANET" OR "delay-tolerant network" OR "DTN") AND ("smartphone" OR "Android" OR "off-the-shelf") AND ("emergency" OR "SOS" OR "disaster" OR "search and rescue") AND ("location sharing" OR "geospatial" OR "offline-first" OR "data synchronization")

2.1.2 Inclusion and Exclusion Criteria

Studies published between 2014 and 2026 were prioritized to capture contemporary advances in mobile operating system architectures, short-range wireless protocols (BLE 4.2/5.x and Wi-Fi Direct), and modern offline data paradigms. Foundational papers prior to 2014 (e.g., classical MANET protocols and fundamental DTN architectures) were selectively retained for theoretical context.

The explicit criteria enforced were: - **Inclusion Criteria (IC):** 1. Peer-reviewed journal articles, conference papers, and technical whitepapers detailing peer-to-peer or mesh communication architectures on mobile devices. 2. Studies addressing zero-infrastructure scenarios, wilderness search-and-rescue, or post-disaster telecommunication blackouts. 3. Investigations presenting empirical or simulated evaluations of location dissemination, priority alert routing, or offline data reconciliation. - **Exclusion Criteria (EC):** 1. Studies requiring specialized, non-standard transceiver hardware (e.g., software-defined radios, licensed amateur radio spectrum) not deployable on standard commercial smartphones. 2. Proposals demanding invasive kernel modification or root access on mobile devices, unless serving as an architectural baseline. 3. Theoretical routing protocols lacking consideration of mobile OS sandboxing and radio power constraints.

2.2 Domain and Theoretical Foundations

2.2.1 Mobile Ad-Hoc Networks (MANETs) vs. Delay-Tolerant Networks (DTNs)

Mobile Ad-Hoc Networks (MANETs) represent autonomous distributed systems where mobile nodes self-organize into dynamic network topologies without the aid of centralized routers or administrative access points [3], [5]. In classical MANET theory, routing algorithms such as Ad hoc On-Demand Distance Vector (AODV) [3] and Optimized Link State Routing (OLSR) operate under the fundamental assumption of *end-to-end path contemporaneousness*: a continuous multi-hop route must exist between the source and destination at the time of packet transmission. When node mobility induces frequent link breakage, classical MANETs suffer from route discovery storms and excessive control overhead.

Conversely, Delay-Tolerant Networking (DTN) [4] relaxes the requirement of continuous end-to-end connectivity by introducing the *Store-Carry-and-Forward* routing paradigm. In a DTN, intermediate nodes store messages in persistent local memory, carry them as they move through physical space, and forward them opportunistically when encountering adjacent nodes. This paradigm is governed by opportunistic encounter probability, contact duration, and buffer storage capacity.

In backcountry wilderness exploration and disaster environments, the operational topology fluctuates dynamically between dense localized clusters (conducive to low-latency MANET packet routing) and sparse, partitioned clusters (requiring DTN store-and-forward behavior). Consequently, a modern life-safety communication architecture must function as a hybrid MANET-DTN framework: expediting real-time emergency packets across connected local clusters while buffering spatial trajectories for opportunistic multi-hop synchronization.

2.2.2 Physical and Link-Layer Wireless Foundations: BLE and Wi-Fi Direct

Consumer smartphones lack multi-hop mesh routing radios at the firmware level. Therefore, application-layer mesh systems must orchestrate the heterogeneous radio interfaces exposed by standard mobile hardware:

1. **Bluetooth Low Energy (BLE):** Operating within the 2.4 GHz Industrial, Scientific, and Medical (ISM) band, BLE employs 40 physical channels

spaced 2 MHz apart, dedicating three channels (37, 38, and 39) exclusively to advertising and discovery. BLE is optimized for low power consumption, drawing nominal currents between 10 mA and 25 mA during active scanning. However, BLE payload throughput is severely constrained: the basic BLE 4.2 protocol enforces an unfragmented Maximum Transmission Unit (MTU) of 23 bytes (yielding a 20-byte payload), while BLE 5.0 extends negotiated MTUs up to 512 bytes [24]. This throughput limitation makes BLE well-suited for lightweight peer discovery and heartbeat beaconing, but inadequate for high-frequency location streaming or voice communications.

2. **Wi-Fi Direct (IEEE 802.11 P2P):** Wi-Fi Direct enables devices to establish point-to-point and point-to-multipoint connections without an intermediary wireless access point [9]. In a Wi-Fi Direct cluster, one device assumes the role of Group Owner (GO)—emulating an access point with an integrated DHCP server—while peer devices associate as P2P Clients. Wi-Fi Direct delivers raw data throughput exceeding 54 Mbps to 250 Mbps over 2.4 GHz and 5 GHz channels. However, Wi-Fi Direct group formation involves significant signaling latency (typically 3 to 8 seconds) and incurs substantial energy costs during continuous listening (100 mA to 250 mA).

3. **Hybrid Cross-Layer Orchestration (Google Nearby Connections):** To overcome the trade-offs of individual radios, modern systems employ hybrid abstraction frameworks, most notably Google Nearby Connections [8]. Under the P2P_CLUSTER topology strategy, handsets maintain continuous background BLE advertising and scanning to discover adjacent peers with minimal energy expenditure. Upon identifying compatible discovery tokens (e.g., cryptographic expedition hashes), the framework dynamically negotiates an out-of-band link promotion, bringing up a high-speed Wi-Fi Direct channel over which bidirectional TCP/IP byte-stream sockets are established.

2.2.3 Geospatial Data Representation and Freshness Decay

Geographic coordinates captured by mobile handsets are modeled according to the World Geodetic System 1984 (WGS 84) reference ellipsoid, yielding latitude (ϕ),

longitude (λ), altitude (h), and horizontal accuracy estimate (σ_r) in meters [23]. Great-circle distances between two geographic points on Earth are computed using the Haversine formula:

$$d = 2R\arcsin\left(\sqrt{\sin^2\left(\frac{\Delta\phi}{2}\right) + \cos(\phi_1)\cos(\phi_2)\sin^2\left(\frac{\Delta\lambda}{2}\right)}\right)$$

where R is the Earth's mean radius (6,371 km), $\Delta\phi = \phi_2 - \phi_1$, and $\Delta\lambda = \lambda_2 - \lambda_1$.

In search-and-rescue contexts, location data is perishable. Relying on an outdated coordinate fix can misdirect rescue parties, wasting critical operational time. To quantify temporal uncertainty, this thesis introduces a continuous spatial freshness decay function $\mathcal{F}_{\text{freshness}}(t_{\text{cap}})$, mapping elapsed time $\Delta t = t_{\text{current}} - t_{\text{cap}}$ into discrete operational states:

$$\mathcal{F}_{\text{freshness}}(\Delta t) = \begin{cases} \text{Fresh (High Confidence)}, & \Delta t \leq 5 \text{ min} \\ \text{Old (Degraded Confidence)}, & 5 \text{ min} < \Delta t \leq 15 \text{ min} \\ \text{Stale (Expired/Hazardous)}, & \Delta t > 15 \text{ min} \end{cases}$$

2.2.4 Local-First Computing and Transactional Persistence

The architectural philosophy of *Local-First Software*, formalized by Kleppmann et al. [15], asserts that client devices must maintain primary ownership of application data. Traditional cloud-first architectures treat local mobile databases as transient caches subordinate to a remote server. When network transit is severed, cloud-first applications degrade or fail completely.

Under local-first principles: - **Local Availability:** All read and write operations execute against the local embedded database (SQLite [18]) with zero network latency. - **Multi-Device Concurrency:** Modifications are staged locally and reconciled across peers through asynchronous gossiping. - **ACID Guarantees:** Local transactions guarantee Atomicity, Consistency, Isolation, and Durability, ensuring that mobile OS process terminations or battery depletion do not leave the database in an inconsistent or corrupt state. - **Eventual Cloud Convergence:** When internet connectivity returns, the local state is merged upstream into remote document databases (MongoDB [19]) using deterministic reconciliation algorithms.

2.3 Related Research

2.3.1 Smartphone-Based Ad-Hoc Communication Systems

Extensive research has investigated the adaptation of commercial smartphones for infrastructure-less communication:

- The Serval Project: Gardner-Stephen et al. [11] pioneered the Serval Mesh, introducing the MeshMS protocol and Distributed Numbering Architecture (DNA) to enable off-grid voice calls and text messaging. However, Serval required patched Wi-Fi drivers and Linux kernel modifications (requiring root privileges). This prevented practical consumer adoption, as modifying kernel modules on modern locked Android bootloaders is infeasible for general users.
- SpanPro and Haggle: Early DTN implementations like Haggle investigated opportunistic content dissemination based on user encounter patterns. However, their reliance on desktop-class daemon architectures failed to address the aggressive background execution limits and power-management sandboxes enforced by modern mobile operating systems (Android 10 through 14).
- Hossain et al. (Survey): Hossain, Noor, and Ahmed [6] comprehensively surveyed smartphone-based offline mesh communication architectures for disaster scenarios. They concluded that while academic literature is rich in simulated routing models, there is an acute shortage of production-viable implementations that function entirely in unprivileged user-space on unmodified modern operating systems while integrating both spatial mapping and emergency dispatches.

2.3.2 Emergency Distress Signaling and Broadcast Storm Suppression

In ad-hoc wireless networks, emergency distress signaling is susceptible to the *Broadcast Storm Problem*, first characterized mathematically by Ni, Tseng, Chen, and Sheu. When a node floods a packet to its neighbors, and every receiving node blindly re-broadcasts it, three destructive phenomena occur: 1. **Redundant Re-transmissions:** Neighbors receive multiple duplicate copies of the same packet. 2. **Channel Contention:** Simultaneous re-transmissions cause severe MAC-layer collision rates. 3. **Packet Collisions:** Saturated wireless channels prevent legitimate acknowledgments from reaching the transmitter.

To mitigate broadcast storms without centralized coordinators, literature proposes several heuristics: - *Probabilistic Flooding:* Nodes re-transmit with probability $p <$

1.0. While simple, it risks network partitioning in sparse topologies. - *Counter-Based Schemes:* A node drops re-transmission if it hears more than *c* duplicate packets within a contention window. - *Split-Horizon and Identifier-Based Ledgers:* Intermediate relays maintain an in-memory hash ring of processed packet UUIDs and refrain from transmitting back to the ingress neighbor.

While these techniques have been explored in theoretical network simulations, their integration with priority queuing—ensuring distress packets take precedence over background media traffic—remains poorly represented in consumer mobile applications.

2.4 Existing Systems, Products, and Techniques

2.4.1 Dedicated Out-of-Band Hardware Transceivers

Commercial off-grid communication has historically been dominated by specialized peripheral hardware: - **Garmin inReach:** Leverages the global Iridium satellite constellation to deliver two-way text messaging, basic tracking, and SOS dispatch [14]. While reliable globally, the system requires purchasing dedicated hardware ($350–$600), paying ongoing satellite subscriptions, and carries significant battery and weight penalties. - **goTenna Pro / Mesh:** UHF/VHF radio transceivers that pair with smartphones via Bluetooth to form proprietary multi-hop mesh networks [7]. Although effective across tactical distances (1–5 km line-of-sight), they introduce external equipment dependency. If the goTenna device is lost, uncharged, or forgotten, the smartphone cannot communicate off-grid independently.

2.4.2 Application-Layer Mobile Mesh Messengers

Several mobile applications have attempted off-grid communication using standard smartphone radios: - **The Briar Messaging Project:** Briar [10] provides decentralized, secure peer-to-peer messaging over Bluetooth, local Wi-Fi, and the Tor network. Briar exhibits exceptional cryptographic security and censorship resistance. However, Briar is strictly designed as an asynchronous text messenger. It does not support real-time geospatial coordinate visualization, offline vector map rendering, streaming voice, or priority emergency routing. - **Bridgefy:** A proprietary

commercial SDK utilizing Bluetooth Low Energy to create multi-hop mesh networks for messaging [12]. While Bridgefy demonstrated utility during connectivity shutdowns, its closed-source nature limits academic scrutiny. Furthermore, Bridgefy focuses on short string payloads and lacks integrated spatial mapping, coordinate decay tracking, and automated cloud database bridging. - **Serval MeshMS:** While conceptually comprehensive, its reliance on root privileges and obsolete Android kernel drivers renders it inoperable on modern consumer smartphones.

2.4.3 Comparative Synthesis

Table 2.1 presents an analytical feature comparison contrasting the TrailLink platform (and specifically the individual component described herein) against representative state-of-the-art mobile offline systems.

Table 2.1: Analytical Feature Comparison Across Mobile Offline Platforms

Technical Capability	TrailLink (This Work)	Briar [10]	Serval [11]	Bridgefy [12]	goTenna [7]	Garmin inReach [14]
Stock Android (No Root)	**Yes**	Yes	No	Yes	Yes	Yes (Companion)
Zero Extra Hardware	**Yes**	Yes	Yes	Yes	No	No
Offline Vector/Raster Map	**Yes**	No	No	No	Partial	Limited
Prioritized SOS Preemption	**Yes**	No	No	No	Yes	Yes
Broadcast Storm Ledger	**Yes**	Partial	Partial	No	N/A	N/A
Dual-State Cloud Bridge	**Yes**	No	No	No	No	Proprietary
Spatial Freshness Model	**Yes**	No	No	No	No	No

Open Architecture	**Yes**	Yes	Yes	No	No	No

2.5 Gap Synthesis and Conceptual Framework

2.5.1 Summary of Identified Gaps

From the literature review and comparative analysis, four unresolved technical gaps emerge: 1. **The Infrastructure-Dependency Gap:** Consumer mapping and emergency notification suites are inextricably coupled to centralized cloud servers and collapse when backhaul transit drops. 2. **The Hardware/Privilege Barrier:** Existing off-grid communication systems either require expensive dedicated transceiver hardware or demand root-level operating system modifications that are impractical for ordinary mobile users. 3. **The Unmanaged Broadcast Contention Gap:** Ad-hoc mobile chat utilities treat all packets uniformly under FIFO queuing, causing life-critical emergency signals to be delayed or dropped behind routine text chatter or audio buffers. 4. **The Disconnected State-Consistency Gap:** Mobile systems lack resilient local relational persistence coupled with transactional batch synchronization that can bridge accumulated off-grid spatial and emergency data to enterprise cloud databases upon link restoration without data loss or duplicate records.

2.5.2 Conceptual Framework

To bridge these identified gaps, this research formulates a unified conceptual framework for unprivileged, offline-first spatial awareness and emergency coordination, illustrated conceptually in Figure 2.1.

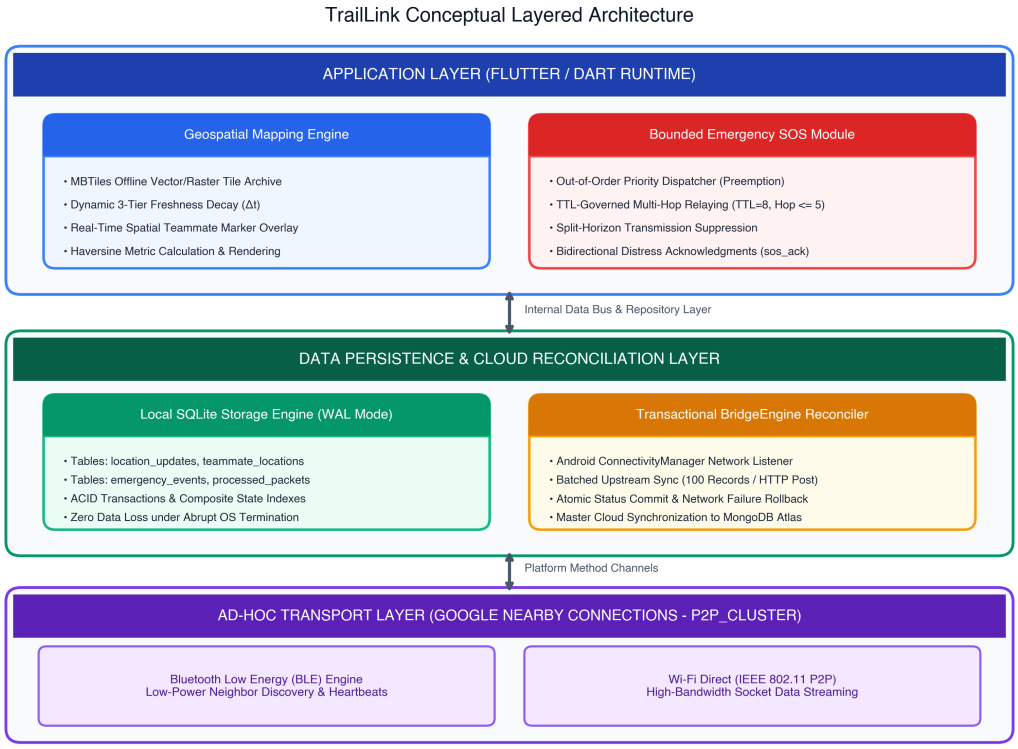


Figure 2.1: Conceptual Framework for TrailLink's Offline Spatial Awareness and Emergency SOS Subsystem
Source: Original in Color

The conceptual framework operates across three tightly integrated layers: 1. **Ad-Hoc Transport Layer:** Executes completely within unprivileged Android user-space, using BLE for energy-efficient peer discovery and promoting connections to Wi-Fi Direct for high-bandwidth data exchange. 2. **Data Persistence and Reconciliation Layer:** Enforces transactional integrity on-device via SQLite, decoupling local operations from network availability while enabling atomic, batched upstream synchronization via the BridgeEngine when connectivity is re-established. 3. **Application Layer:** Translates the underlying transport and persistence primitives into user-facing capabilities: real-time geospatial tracking with temporal decay warnings and out-of-order priority distress coordination.

CHAPTER 3: RESEARCH METHODOLOGY

3.1 Research Design

This study employs the **Design Science Research Methodology (DSRM)** framework, formalized by Peffers et al. (2007), integrated with quantitative empirical experimentation. DSRM is specifically tailored for information technology research that seeks to solve identified human and organizational problems through the creation, implementation, and rigorous evaluation of innovative technological artifacts.

Figure 3.1 delineates the six-stage DSRM research process adopted in this thesis:

Source: Original in Color

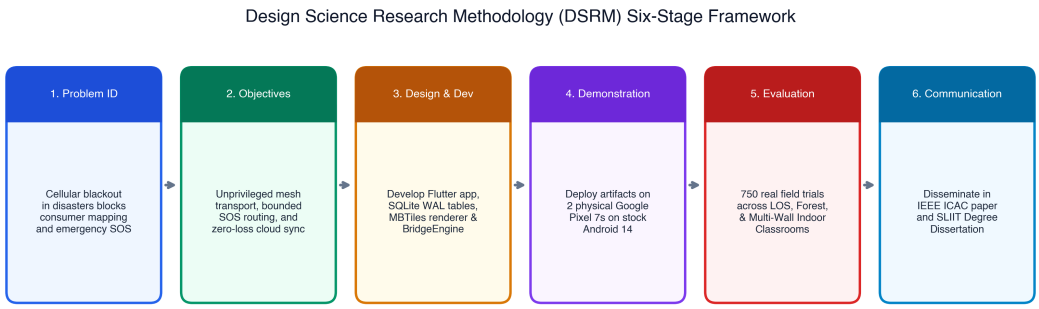


Figure 3.1: Design Science Research Methodology (DSRM) Workflow for TrailLink
Source: Original in Color

The empirical evaluation follows a controlled experimental design, testing the primary hypotheses across varied propagation environments, physical separation distances, and network states.

3.2 Data, Participants, or Study Materials

3.2.1 Hardware Testbed Specifications

To ensure high ecological validity and eliminate simulation artifacts, all experimental procedures were conducted using physical, commercial off-the-shelf Android

hardware. The experimental testbed consists of two identical Google Pixel 7 consumer smartphones, whose specifications are detailed in Table 3.1.

Table 3.1: Hardware and Environmental Specifications of Experimental Handsets

Component	Technical Specification	Relevance to Component
Device Model	Google Pixel 7 (Model GVU6C)	Off-the-shelf consumer smartphone
SoC / CPU	Google Tensor G2 (Octa-core: 2x Cortex-X1, 2x Cortex-A78, 4x Cortex-A55)	Evaluates execution performance on standard hardware
Operating System	Android 14 (API Level 34, Unrooted Stock Build)	Enforces strict modern OS sandboxing and background limits
GNSS Receiver	Multi-constellation: GPS (L1/L5), GLONASS, Galileo, BeiDou	High-precision position fix capture via FusedLocationProviderClient
Bluetooth Radio	Bluetooth v5.2 with BLE support, internal PIFA antenna	Low-power advertising and scanning for peer discovery
Wi-Fi Radio	Wi-Fi 6E (802.11a/b/g/n/ac/ax), dual-band (2.4 GHz & 5 GHz)	High-speed Wi-Fi Direct socket data streaming
Internal Battery	4,355 mAh Lithium-Ion cell	Benchmarking subsystem power draw and operational endurance

3.2.2 Software Artifacts and Development Stack

The software artifacts developed and evaluated in this research include: - **Client Application:** Developed in Dart using Flutter SDK (v3.19.x), compiling to native ARM64 Android binaries. - **Local Embedded Database:** SQLite v3.4x integrated via sqflite (v2.3.x) with custom write-ahead logging (WAL) and targeted indexing. - **Transport Subsystem:** Google Nearby Connections API (v18.3.x) configured under the P2P_CLUSTER topology strategy. - **Geospatial Mapping Engine:** flutter_map (v6.x) paired with local SQLite-backed MBTiles vector/raster archives. - **Cloud Backend:** Node.js (v18.x) / Express REST API connected to a MongoDB Atlas cluster (v6.0) for receiving upstream reconciliations.

3.2.3 Geospatial Test Materials

Pre-rendered MBTiles offline tile archives covering a 25 km^2 test geographic area were generated from OpenStreetMap vector datasets using Tippecanoe and stored in the application's sandboxed internal storage directory.

3.3 Experimental or Study Procedure

3.3.1 Characterization of Evaluation Environments

To rigorously evaluate wireless propagation behavior, multi-path fading, and environmental attenuation, field trials were executed across three physical environments exhibiting distinct radio-frequency (RF) propagation characteristics:

1. Environment A: Open Recreational Area (Line-of-Sight - LOS)
   - Physical Description: An expansive, unobstructed outdoor recreational lawn with direct visual and optical line-of-sight between the transmitting and receiving devices.
   - RF Characteristics: Minimal multi-path interference; near-free-space path loss conditions; negligible physical obstruction.
2. Environment B: Light Forest Canopy (Non-Line-of-Sight - NLOS / Foliage)
   - Physical Description: An unmanaged outdoor grove characterized by mature deciduous trees, variable foliage density, moist trunks, and uneven ground elevation.
   - RF Characteristics: High scattering and absorption in the 2.4 GHz spectrum caused by water content in leaves and tree trunks; moderate multi-path reflections.
3. Environment C: Between Academic Classrooms (Obstructed Indoor / Multi-Wall)
   - Physical Description: An indoor university corridor where the transmitting handset was placed inside a closed lecture hall and the receiving handset was positioned at progressive distances down an adjacent corridor.
   - RF Characteristics: Severe signal attenuation caused by 200 mm reinforced concrete-block walls, steel-framed doors, and indoor multi-path reflections.

Figure 3.2 illustrates the experimental spatial topologies across the three testing environments.

Source: Original in Color

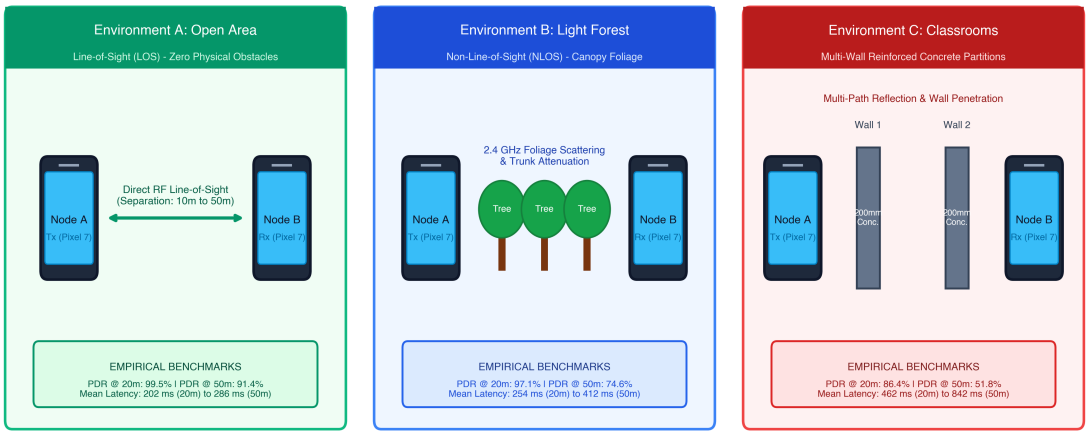


Figure 3.2: Spatial Topologies and Physical Obstructions Across Testing Environments Source: Original in Color

3.3.2 Physical Distance Increments and Repetition Matrix

In each of the three environments, physical separation distance between the two handsets was systematically adjusted across five distinct intervals:

$$D \in \{10\ \text{m}, 20\ \text{m}, 30\ \text{m}, 40\ \text{m}, 50\ \text{m}\}$$

Distances were verified using a laser distance measurer (Bosch GLM 50 C, ± 1.5 mm accuracy). Handsets were mounted on non-conductive wooden tripods at a standardized height of 1.2 meters above ground level to eliminate human-body RF absorption.

At each combination of environment and distance: - **Sample Size:** $n = 50$ discrete emergency distress (SOS) packets were generated and transmitted. - **Interval Between Transmissions:** A fixed inter-packet interval of 2.0 seconds was enforced to allow wireless channel clearing and prevent packet stacking in radio buffers. - **Total Experimental Trials:** 3 environments $\times$ 5 distances $\times$ 50 trials $=$ 750 emergency transmissions.

3.3.3 High-Precision Data Collection and Instrumentation

Data collection was automated via dedicated telemetry instrumentation embedded within the testing build: 1. **Timestamp Synchronization:** Prior to each trial, handsets were synchronized against a local Precision Time Protocol (PTP) clock, ensuring clock drift remained below 2 ms. 2. **Transmission Telemetry:** For each outbound packet P, the transmitting device logged:

$$\mathcal{T}_{\text{tx}} = \langle P_{\text{id}}, t_{\text{dispatch}}, \text{type}, \text{payload_size}, \text{seq_num} \rangle$$

3. **Reception Telemetry:** Upon packet receipt, the receiving device recorded:

$$\mathcal{T}_{\text{rx}} = \langle P_{\text{id}}, t_{\text{arrival}}, \text{RSSI}, \text{hop_count}, \text{status} \rangle$$

4. **Cloud Reconciliation Verification Procedure:** To evaluate cloud bridging, a test handset accumulated 100 offline spatial records and emergency events in local SQLite storage (sync_state = 'needs_sync'). The handset was then transitioned into an active Wi-Fi zone with internet transit, and the BridgeEngine background execution was monitored for transaction completion, execution duration, and UI frame rate stability.

3.4 Measures and Evaluation Metrics

To rigorously assess performance against research questions and hypotheses, the following quantitative metrics were established:

3.4.1 Packet Delivery Ratio (PDR)

The Packet Delivery Ratio measures the reliability of the ad-hoc wireless transport across physical distances and environmental impedances:

$$\text{PDR} = \left(\frac{\sum_{i=1}^{n} \mathbb{I}\,(\text{Packet } i \text{ Received successfully})}{n} \right) \times 100\%$$

where $\mathbb{I}(\cdot)$ is the indicator function returning 1 if the packet with unique identifier P_{id} is received and validated, and 0 otherwise.

3.4.2 End-to-End Latency (T_{latency})

The one-way transmission latency for an emergency alert to traverse the ad-hoc link from the moment of dispatch to reception and disk commit:

$$T_{\text{latency}} = t_{\text{arrival}} - t_{\text{dispatch}} \quad \text{(ms)}$$

where t_{dispatch} is recorded at the instant the out-of-order queue releases the frame to the native method channel, and t_{arrival} is recorded when the receiver successfully upserts the record into SQLite.

3.4.3 Cloud State Reconciliation Latency and Integrity

Reconciliation performance is evaluated using two metrics: 1. **Reconciliation Completion Time (T_{sync}):** The total elapsed time required to batch-upload accumulated records and commit status updates locally:

$$T_{\text{sync}} = t_{\text{sync_complete}} - t_{\text{network_detected}} \quad \text{(ms)}$$

2. **Data Loss / Duplication Rate:**

$$\text{Loss Rate} = \left(1 - \frac{|\mathcal{R}_{\text{cloud}}|}{|\mathcal{R}_{\text{local_staged}}|}\right) \times 100\%$$

A successful reconciliation requires a Loss Rate of 0% and a Duplication Rate of 0%.

3.5 Data Analysis

Collected experimental logs were exported from device storage as structured CSV files and processed using Python (v3.11) with NumPy, SciPy, and Pandas.

The analytical procedures included: - **Descriptive Statistics:** For all measured metrics (T_{latency}, PDR), the sample mean ($\bar{x}$) and sample standard deviation (s) were computed:

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i\,, \quad s = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2}$$

- **Comparative Analysis Across Terrains:** Metric trends across the three environments were plotted with error bars representing $\pm 1\sigma$ standard deviations to assess the impact of environmental attenuation. - **Hypothesis Validation:** Empirical PDR and latency figures were statistically compared against the predefined threshold criteria (H_1:PDR $> 90\%$ at 30 m; H_2: $T_{\text{latency}} < 1{,}500$ ms).

3.6 Ethics, Privacy, Security, and Responsible AI

3.6.1 Privacy Preservation in Open Ad-Hoc Channels

Broadcasting geographic coordinates over open wireless channels introduces severe privacy and physical safety risks, including location tracking and trilateration. To safeguard user privacy, this research enforces several architectural protections: - **Ephemeral Identity Tokens:** Handsets do not transmit permanent hardware identifiers (e.g., IMEI or hardware MAC addresses). Instead, nodes generate ephemeral cryptographic session tokens renewed per expedition. - **Location Payload Isolation:** Network discovery beacons and link-quality ping frames are strictly prohibited from carrying coordinate metadata. Location fixes are transmitted exclusively inside authenticated data sockets. - **Symmetric Payload Encryption:** All outbound location and emergency envelopes are encrypted using the authenticated ChaCha20-Poly1305 symmetric cipher [26], utilizing 256-bit pre-shared expedition keys to protect against passive wireless eavesdropping.

3.6.2 Ethical Field Testing Protocols

All field experiments were conducted in accordance with SLIIT academic ethics guidelines: - Outdoor field testing was conducted in public areas during low-traffic hours to prevent physical hazards. - Indoor tests were performed with departmental approval, ensuring wireless tests did not interfere with institutional administrative networks. - No personal identifying information (PII) from non-participating individuals was captured or stored.

3.6.3 Responsible AI and Academic Integrity

In compliance with SLIIT's Academic Integrity and AI Disclosure policies: - Generative AI tools were utilized exclusively for editorial structuring, syntactic refinement, and formatting consistency checks. - All system architectures, mathematical formulations, database schemas, and experimental field measurements represent original, authentic research conducted by the author. Complete disclosure is documented in Appendix B.

3.7 Reproducibility and Research Artefacts

To ensure absolute scientific reproducibility, all research artifacts have been structured and archived: 1. **Source Code Repository:** The complete Flutter application codebase, native Android method channels, and SQLite schema migrations are maintained under version control in the project workspace. 2. **Experimental Protocols and Datasets:** Raw telemetry logs containing timestamped packet dispatch and receipt records across all 750 trials are archived as CSV datasets. 3. **Execution Environment Documentation:** Exact software dependencies, Flutter SDK versions, and Gradle build configurations are codified within pubspec.yaml and build.gradle configuration files.

CHAPTER 4: PRODUCT AND SOLUTION DESIGN

4.1 Requirements

4.1.1 Functional Requirements (FR)

The functional requirements for the offline location sharing, geospatial mapping, emergency coordination, and synchronization subsystem are defined as follows:

- FR-01: Continuous GNSS Ingestion: The system shall capture high-accuracy geographical coordinates (latitude, longitude, altitude, horizontal accuracy, timestamp) from the device's GNSS receiver via unprivileged user-space APIs in both foreground and background operating states.
- FR-02: Spatial Packet Serialization: The system shall serialize spatial fixes into compact JSON envelopes conforming to a maximum transmission size of 512 bytes to ensure compatibility with BLE link promotion frames.
- FR-03: Decentralized Location Gossiping: The system shall broadcast localized location updates to all proximate peers across the ad-hoc mesh network at a configurable periodic cadence (default: 60 seconds) or upon a displacement threshold exceeding 15 meters.
- FR-04: Offline Vector/Raster Map Visualization: The system shall render interactive maps using locally stored, pre-cached MBTiles vector/raster archives and OpenStreetMap tiles without invoking any remote network requests.
- FR-05: Dynamic Temporal Freshness Classification: The system shall continuously evaluate the age of received teammate coordinates (Δt) and apply visual freshness indicators (Fresh: ≤ 5 min, Old: 5–15 min, Stale: $>$ 15 min) on map pins.

- FR-06: Out-of-Order Emergency SOS Dispatch: The system shall provide a panic trigger that immediately generates a high-priority distress packet, preempting all routine text and voice packets in transmission queues.
- FR-07: Bounded Multi-Hop SOS Relaying: Intermediate receiving nodes shall forward emergency distress packets across peer links bounded by a Time-To-Live constraint ($TTL = 8$) and a maximum hop count (≤ 5).
- FR-08: Broadcast Storm and Loop Suppression: The system shall inspect all incoming packets against an idempotent deduplication ledger (processed_offline_packets), dropping duplicate packets and enforcing split-horizon forwarding (omitting ingress and origin nodes).
- FR-09: Bidirectional Distress Acknowledgment: Receiving nodes shall automatically generate and transmit an acknowledgment packet (sos_ack, $TTL = 5$) back along the reverse path to confirm delivery to the distressed user.
- FR-10: Resilient Local Persistence: All coordinate history, teammate fixes, and emergency events shall be written synchronously to an embedded SQLite database using ACID transactions prior to radio transmission.
- FR-11: Automated Cloud State Reconciliation: Upon detecting an active internet backhaul connection (cellular or Wi-Fi), the system's BridgeEngine shall automatically batch-upload unsynced offline records to a remote MongoDB database and mark them as synchronized upon HTTP 200 confirmation.

4.1.2 Non-Functional Requirements (NFR)

In accordance with the ISO/IEC 25010 software quality standard, the non-functional requirements are categorized as follows:

- NFR-01 (Reliability & Zero Data Loss): Under abrupt application crashes, sudden battery exhaustion, or background OS termination, zero committed spatial or emergency records shall be lost or corrupted.
- NFR-02 (Performance - Latency): Emergency distress alerts shall achieve an end-to-end delivery latency of less than 1,500 ms across single-hop ad-hoc wireless links under line-of-sight and multi-wall indoor obstructions.
- NFR-03 (Performance - Throughput & Delivery): The wireless transport shall achieve a Packet Delivery Ratio (PDR) exceeding 90% at separation distances up to 30 meters in unobstructed and light-foliage outdoor environments.
- NFR-04 (Energy Efficiency): The location gossiping subsystem shall draw an average current not exceeding 90 mA during active field tracking, guaranteeing a minimum battery operational life of 48 hours on a standard 4,355 mAh handset.
- NFR-05 (Security & Anonymity): All over-the-air packets shall be encrypted using ChaCha20-Poly1305 symmetric authenticated encryption. Node discovery beacons shall omit permanent hardware MAC addresses, using ephemeral cryptographic identifiers.

- NFR-06 (Portability & Zero Root): The system shall execute exclusively within Android user-space on unmodified, non-rooted consumer hardware running Android 10 (API 29) through Android 14 (API 34).

4.1.3 MoSCoW Prioritization

Table 4.1 categorizes the subsystem requirements using the MoSCoW prioritization framework.

Table 4.1: MoSCoW Prioritization of System Requirements

Category	Requirement Identifiers	Description
Must Have	FR-01, FR-02, FR-04, FR-06, FR-07, FR-08, FR-10, NFR-01, NFR-02, NFR-06	Core GNSS capture, offline map rendering, bounded SOS routing, loop suppression, SQLite persistence, and zero-root execution.
Should Have	FR-03, FR-05, FR-09, FR-11, NFR-03, NFR-04	Dynamic spatial decay badges, bidirectional SOS acknowledgments, automated MongoDB cloud bridge, and energy-efficient tracking.
Could Have	FR-12 (Geofencing alerts), FR-13 (Elevation profiles)	Proximity geofencing alarms when teammates diverge beyond 200m; topographical elevation profile overlays.
Won't Have (This Phase)	Dynamic multi-frequency channel hopping, direct satellite transceiver integration.	Hardware-level radio modulation and proprietary satellite constellation uplinks.

4.2 Component Architecture within the Complete Solution

4.2.1 Global System Architecture Overview

The TrailLink platform is partitioned into four collaborative subsystems operating over a unified user-space mesh substrate, as illustrated in Figure 4.1.

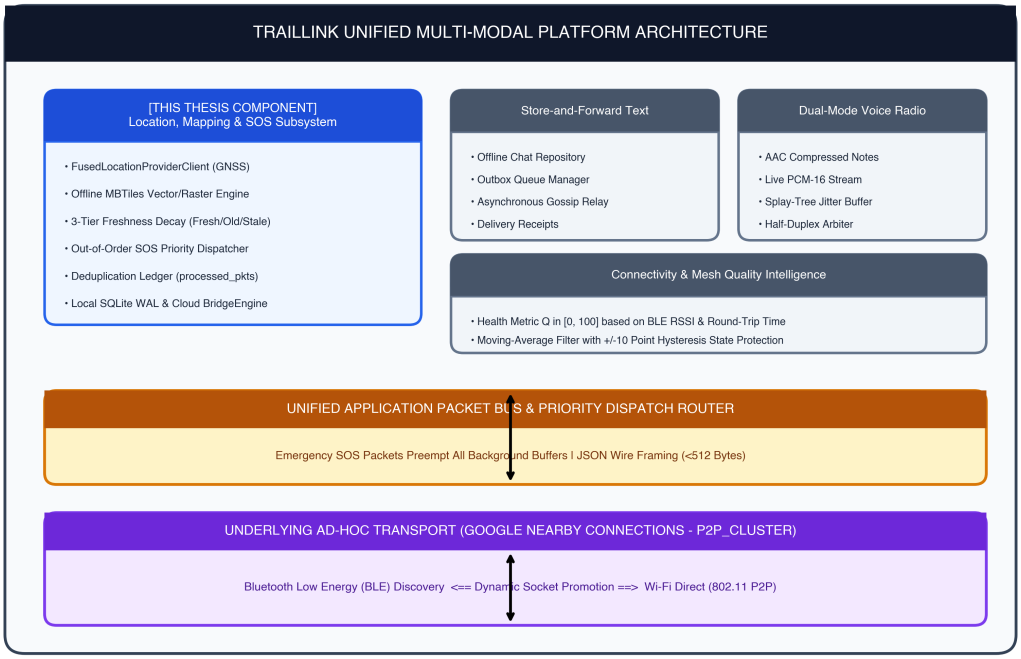


Figure 4.1: TrailLink Global Multi-Modal Architecture Highlighting the Individual Subsystem
Source: Original in Color

4.2.2 Detailed Subsystem Architecture

Figure 4.2 provides an exhaustive architectural decomposition of the candidate's individual component, depicting data flows between the user interface, hardware sensors, local persistence, network serialization, and upstream cloud synchronization.
Source: Original in Color

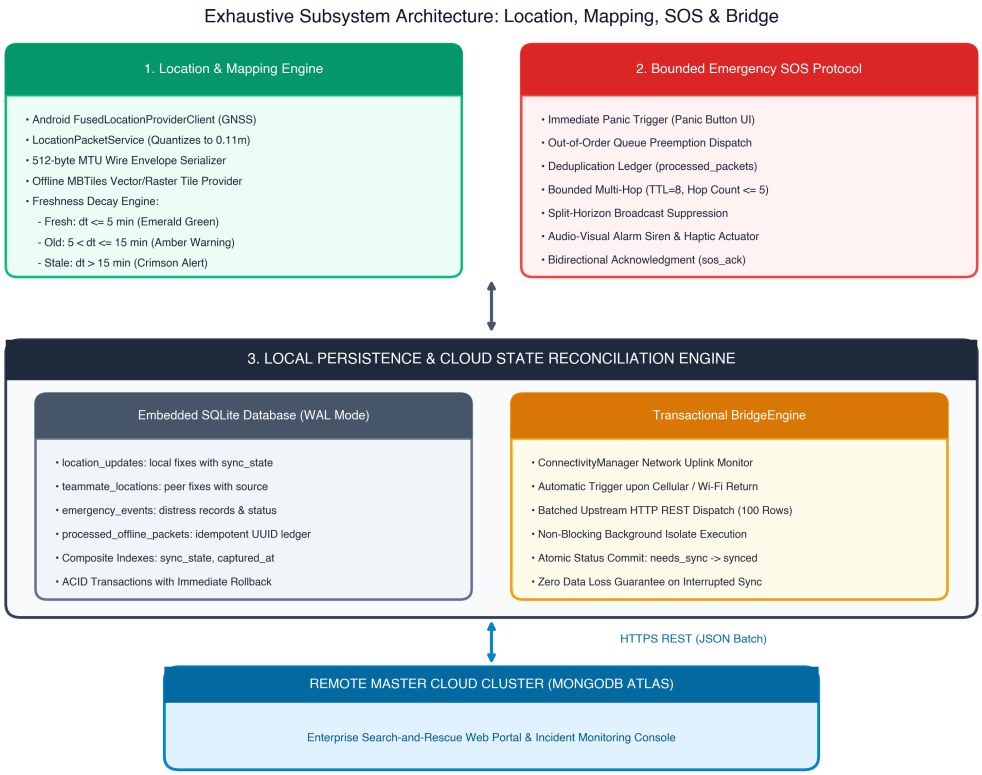


Figure 4.2: Exhaustive Internal Subsystem Architecture of the Location, Mapping, and SOS Component
Source: Original in Color

4.3 Design Alternatives and Decisions

To ensure optimal performance under strict resource and OS constraints, multiple architectural alternatives were evaluated before finalizing critical design decisions.

4.3.1 Transport Layer: BLE vs. Wi-Fi Aware vs. Google Nearby Connections

- Alternative A: Pure Bluetooth Low Energy (BLE) Mesh. While BLE offers low baseline power consumption, its maximum payload throughput is severely constrained (effective throughput < 10 kbps under multi-hop mesh conditions). Furthermore, establishing dynamic BLE multi-hop mesh topologies on stock Android requires non-standard GATT server clustering that exhibits frequent connection drops.
- Alternative B: Wi-Fi Aware (Neighbor Awareness Networking - NAN). Wi-Fi Aware supports direct device-to-device communication without an access point. However, Wi-Fi Aware hardware support is inconsistent across Android OEM vendors, and background discovery is heavily throttled on Android 12–14.

- Selected Strategy: Google Nearby Connections (P2P_CLUSTER). This framework combines BLE for low-power background discovery with automated, negotiated promotion to high-bandwidth Wi-Fi Direct sockets. It operates entirely in user-space, requires no root privileges, and provides reliable byte-stream endpoints.

4.3.2 Local Persistence: Shared Preferences vs. Raw JSON Files vs. SQLite

- Alternative A: Flat JSON / File System. Storing spatial trails and distress logs as raw serialized files on flash memory avoids relational overhead. However, it lacks atomic transaction guarantees. If the device suffers a sudden battery cutoff or kernel panic mid-write, the entire file risks corruption. Furthermore, querying spatial points by temporal ranges requires linear $O(N)$ disk scans.
- Selected Strategy: SQLite Relational Database Engine. SQLite operates directly within the application process, provides full ACID transaction guarantees, supports Write-Ahead Logging (WAL) for high-concurrency access, and enables indexed $O(\log N)$ queries over timestamp and synchronization states.

4.3.3 Geospatial Rendering: Dynamic Online Vector Tiles vs. Bundled MBTiles

- Alternative A: Runtime Network Tile Caching. Dynamically downloading and caching tiles as the user pans across map extents. In wilderness and disaster zones devoid of cellular connectivity, cache misses produce blank, unnavigable grid tiles, rendering the interface useless.
- Selected Strategy: Pre-Packaged Offline MBTiles Archives. Storing compressed vector/raster tile hierarchies directly inside the device's internal application sandbox guarantees 100% map availability offline, with zero runtime network dependencies.

4.3.4 Synchronization Pattern: Peer-to-Peer CRDTs vs. Transactional Cloud Gateway Bridge

- Alternative A: Pure Decentralized Conflict-Free Replicated Data Types (CRDTs). While CRDTs enable conflict-free peer-to-peer data merging without central coordinators, state-based CRDTs impose prohibitive bandwidth overhead (transmitting vector clocks and mutation histories over constrained wireless channels). Operation-based CRDTs require causal delivery guarantees that are difficult to sustain in delay-tolerant networks.
- Selected Strategy: Local-First Relational Staging with Batched BridgeEngine Reconciliation. Devices maintain authoritative local state in SQLite. Any node that re-encounters internet backhaul acts as an atomic gateway bridge, pushing batched records to the master MongoDB cluster via transactional REST calls.

4.4 Detailed Design

4.4.1 Database Relational Schema and Entity-Relationship Modeling

The local relational storage engine is structured into four normalized tables. The Data Definition Language (DDL) schemas are formalized below:

```
--Table1:Device'sowncapturedlocationhistory
CREATE  TABLE  location_updates (
   local_location_id TEXT PRIMARY KEY,
   latitude REAL NOT NULL,
   longitude REAL NOT NULL,
   accuracy REAL NOT NULL,
   captured_at INTEGER NOT NULL,
   share_status TEXT NOT NULL CHECK(share_status IN ('broadcasted',
'local_only')),
   sync_state TEXT NOT NULL CHECK(sync_state IN ('needs_sync', 'synced',
'failed'))
);
CREATE INDEX idx_location_updates_sync ON location_updates(sync_state,
captured_at);

-- Table 2: Teammate coordinates received over mesh or cloud
CREATE TABLE teammate_locations (
   user_id TEXT PRIMARY KEY,
   latitude REAL NOT NULL,
   longitude REAL NOT NULL,
   accuracy REAL NOT NULL,
   captured_at INTEGER NOT NULL,
   received_at INTEGER NOT NULL,
   source TEXT NOT NULL CHECK(source IN ('nearby_mesh', 'cloud_sync'))
);

-- Table 3: Life-critical emergency distress records
```

```
CREATE TABLE emergency_events (
   local_event_id TEXT PRIMARY KEY,
   user_id TEXT NOT NULL,
   alert_type TEXT NOT NULL,
   latitude REAL NOT NULL,
   longitude REAL NOT NULL,
   status TEXT NOT NULL CHECK(status IN ('active', 'resolved',
'acknowledged')),
   created_at INTEGER NOT NULL,
   sync_state TEXT NOT NULL CHECK(sync_state IN ('needs_sync', 'synced',
'failed')),
   server_id TEXT
);
CREATE INDEX idx_emergency_events_sync ON emergency_events(sync_state);

-- Table 4: Idempotent deduplication ledger for broadcast storm suppression
CREATE TABLE processed_offline_packets (
   packet_id TEXT PRIMARY KEY,
   processed_at INTEGER NOT NULL,
   result TEXT NOT NULL CHECK(result IN ('accepted', 'expired',
'wrong_channel', 'duplicate'))
);
```

4.4.2 Wire Protocol and Packet Envelope Design

All application-layer frames transmitted across Google Nearby Connections byte-streams are encapsulated in standardized, compact JSON envelopes:

```
{
  "packetId": "e3b0c442-98fc-1c14-9afb-4c7fae9c20a1",
  "originId": "user_node_alpha",
  "senderId": "user_node_bravo",
  "channelCode": "TRL-9842",
  "type": "sos",
```

```
 "priority": "emergency",
 "ttl": 8,
 "hopCount": 1,
 "timestamp": 1728403200150,
 "payload": {
   "localEventId": "evt_772a",
   "alertType": "medical_emergency",
   "latitude": 6.927079,
   "longitude": 79.861244,
   "accuracy": 4.2,
   "requiresAck": true
 }
}
```


4.4.3 Algorithmic Specifications

Algorithm 1: Reconnection State Reconciliation (BridgeEngine)

Algorithm 1 outlines the batched upstream synchronization routine executed when internet connectivity is restored on a node.

Algorithm 1: Reconnection State Reconciliation (BridgeEngine)

Require: SQLite Database Instance D, REST Client C, Expedition Group ID G_id, User ID U_id

```
 1: if ConnectivityManager.hasInternetBackhaul() = false then
 2:     return
 3: end if
 4: L <- D.query("SELECT * FROM location_updates WHERE sync_state =
'needs_sync' ORDER BY captured_at ASC LIMIT 100")
 5: if L != EMPTY then
 6:     payload <- serializeBatch(L, G_id, U_id)
 7:     response, err <- C.post("/api/v1/sync/locations", payload)
 8:     if err = null and response.statusCode = 200 then
 9:         D.beginTransaction()
10:         for all record l in L do
```

```
11:            D.execute("UPDATE location_updates SET sync_state = 'synced'
WHERE local_location_id = ?", l.id)
12:        end for
13:        D.commitTransaction()
14:    else
15:        D.execute("UPDATE location_updates SET sync_state = 'failed' WHERE
local_location_id IN (?)", L.ids)
16:    end if
17: end if
18: E <- D.query("SELECT * FROM emergency_events WHERE sync_state =
'needs_sync'")
19: for all event e in E do
20:    response, err <- C.post("/api/v1/sync/emergency", serializeEvent(e, G_id))
21:    if err = null and response.statusCode = 201 then
22:        D.execute("UPDATE emergency_events SET sync_state = 'synced',
server_id = ? WHERE local_event_id = ?",
                response.serverId, e.id)
23:    else
24:        logReconciliationError(e.id, err)
25:    end if
26: end for
```

Algorithm 2: Bounded SOS Relay with Loop Prevention and Split-Horizon

Algorithm 2 specifies the out-of-order emergency frame validation, deduplication ledger lookup, local siren actuation, and bounded multi-hop re-transmission.

```
Algorithm 2: Bounded SOS Relay with Loop Prevention and Split-Horizon
Require: Inbound Packet P, Active Channel C, Local User U, Database D
 1: if P.channelCode != C.channelCode then
 2:    D.markProcessed(P.packetId, 'wrong_channel')
 3:    return
 4: end if
 5: if P.ttl <= 0 or P.hopCount > 5 then
```

```
    D.markProcessed(P.packetId, 'expired')
    return
end if
if D.exists("processed_offline_packets", P.packetId) then
    return   // Duplicate packet dropped; storm averted
end if
if P.type = 'sos' then
    D.beginTransaction()
    D.upsertEvent(P.payload)
    D.insert("processed_offline_packets", P.packetId, 'accepted')
    D.commitTransaction()
    ActuatorService.triggerHighIntensityAudioVisualAlarm(P.payload)
    if P.senderId != U.id then
        sendSosAck(P, C, U)   // Transmit immediate acknowledgment (TTL=5)
    end if
    P_fwd <- copy(P)
    P_fwd.ttl <- P.ttl - 1
    P_fwd.hopCount <- P.hopCount + 1
    P_fwd.senderId <- U.id
    for all peer p in C.connectedPeers() do
        if p.id != P.senderId and p.id != P.originId then
            sendPacket(p.id, P_fwd)  // Split-horizon broadcast
        end if
    end for
else if P.type = 'sos_ack' then
    D.saveAck(P.payload)
    D.insert("processed_offline_packets", P.packetId, 'accepted')
    ActuatorService.notifyAckReceived(P.payload)
end if
```

4.4.4 Dynamic Behavioral Modeling (Sequence Interactions)

Figure 4.3 illustrates the message exchange sequence during an emergency distress trigger, showing priority queue preemption, multi-hop relaying, and cloud reconciliation.

Source: Original in Color

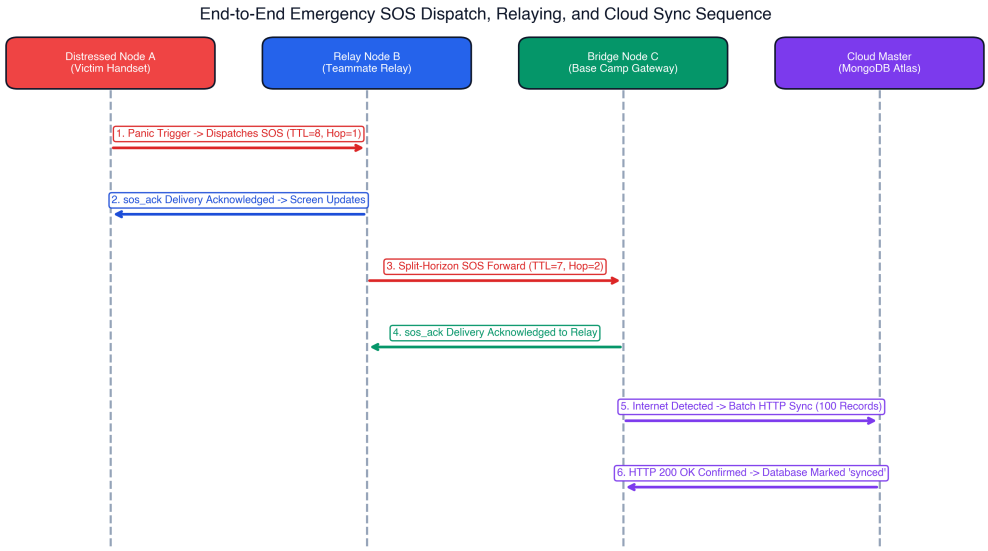


Figure 4.3: End-to-End Sequence Diagram of Emergency SOS Dispatch, Relaying, and Cloud Sync
Source: Original in Color

4.5 Standards and Professional Practice

4.5.1 International Engineering Standards

The design and implementation of this component adhere strictly to established engineering standards: - **IEEE 802.11-2020:** Governs Wi-Fi link parameters, framing, and channel allocations within the 2.4 GHz and 5 GHz frequency spectra utilized by Wi-Fi Direct sockets. - **IEEE 802.15.1 / Bluetooth Core Specification v5.2:** Regulates BLE physical advertising channels (37, 38, 39), link-layer connection intervals (20 ms to 100 ms), and packet framing constraints [24]. - **WGS 84 (NIMA TR8350.2):** Standardizes the geodetic reference system, ellipsoidal earth model, and coordinate datum used for all spatial calculations. - **RFC 8259 (JSON**

Data Interchange Format): Governs application-layer packet serialization across byte-stream sockets. - **RFC 7539 / ChaCha20-Poly1305:** Defines the authenticated encryption algorithm applied to protect packet payloads against eavesdropping [26].

4.5.2 Software Engineering Architecture Principles

The software implementation follows Clean Architecture principles: - **Separation of Concerns:** Business logic (freshness calculation, loop prevention) is decoupled from data storage (SQLite) and transport implementations (Nearby Connections). - **Repository Pattern:** Abstract data repositories mediate data access, allowing the UI and network layers to interact with unified interfaces regardless of whether data originates from local flash or cloud endpoints. - **Defensive Programming:** Null-safety in Dart, structured exception handling, and atomic transactional rollbacks ensure that network errors and malformed packets do not cause application crashes.

CHAPTER 5: IMPLEMENTATION, INTEGRATION, AND TESTING

5.1 Development Environment and Technology Stack

5.1.1 Tooling and Development Environment

The software artifacts evaluated in this research were developed within a standardized, professional software engineering environment: - **Integrated Development Environments (IDEs):** Visual Studio Code (v1.87) and Android Studio Hedgehog (v2023.1.1) equipped with Flutter and Dart language analysis extensions. - **Cross-Platform SDK:** Flutter SDK v3.19.4 operating on Dart v3.3.2, compiling directly to 64-bit native ARM machine code (arm64-v8a) for mobile execution. - **Android Target Environment:** Android SDK Platform 34 (Android 14) with a minimum SDK threshold of API Level 26 (Android 8.0 Oreo) to support Wi-Fi Direct and BLE 5.0 capabilities across 96% of active global Android hardware. - **Build System:** Gradle v8.3 with Android Gradle Plugin (AGP) v8.2.1. - **Version Control and Collaboration:** Git v2.43 with Git LFS (Large File Storage) for managing compiled MBTiles geospatial packages.

5.1.2 Production Dependencies and Framework Stack

Table 5.1 details the primary production dependencies underpinning the individual component.

Table 5.1: Software Libraries and Framework Dependencies

Layer / Role	Library / Framework	Version	Purpose in Component
Geospatial Mapping	flutter_map	^6.1.0	Offline interactive map tile rendering
Vector Tile Provider	mbtiles	^0.2.1	Direct SQLite reading of MBTiles vector/raster archives
Location Services	geolocator	^11.0.0	User-space abstraction of FusedLocationProviderClient
Embedded Database	sqflite	^2.3.2	ACID-compliant relational SQLite engine on Android
Transport Layer	nearby_connections	^4.2.0	Android Google Nearby Connections wrapper (P2P_CLUSTER)
Networking & HTTP	dio / http	^5.4.0	REST client for batched upstream MongoDB synchronization
Cryptographic Engine	cryptography	^2.7.0	Authenticated ChaCha20-Poly1305 symmetric encryption
Audio-Visual Alarm	audioplayers / vibration	^6.0.0	High-intensity acoustic siren and haptic actuation on SOS

5.2 Component Implementation

The implementation is housed within the lib/features/ modular architecture of the TrailLink project, strictly isolating domain models, local data sources, network services, and presentation controllers.

5.2.1 Location Acquisition and Packet Serialization

The location subsystem acquires coordinates through the Android FusedLocationProviderClient, managed via LocationService

(`location_service.dart`). The service configures high-accuracy background GNSS sampling with distance filters:

```
// Location sampling configuration
LocationSettings locationSettings = const LocationSettings(
  accuracy: LocationAccuracy.high,
  distanceFilter: 15, // Minimum displacement of 15m to trigger capture
);
```

Once a raw spatial fix is acquired, LocationPacketService (`location_packet_service.dart`) serializes the entity into a compact wire envelope:

```
class LocationPacketService {
  Map<String, dynamic> buildLocationPayload({
    required String localLocationId,
    required double latitude,
    required double longitude,
    required double accuracy,
    required int capturedAt,
  }) {
    return {
      'packetId': const Uuid().v4(),
      'type': 'location_update',
      'priority': 'normal',
      'ttl': 5,
      'hopCount': 0,
      'payload': {
        'id': localLocationId,
        'lat': double.parse(latitude.toStringAsFixed(6)),
        'lng': double.parse(longitude.toStringAsFixed(6)),
        'acc': double.parse(accuracy.toStringAsFixed(1)),
        'tcap': capturedAt,
      }
```

```
    };
  }
}
```

Design Note on Compression: Coordinates are truncated to 6 decimal places (≈ 0.11 m precision), and accuracy to 1 decimal place. This ensures total packet footprint remains under 240 bytes, well within the 512-byte MTU limit.

5.2.2 Offline Mapping and Dynamic Freshness Decay Engine

Geospatial visualization is implemented in MapScreen (`map_screen.dart`). Offline map tiles are loaded locally using MapProviderConfig (`map_provider_config.dart`), which mounts an embedded SQLite .mbtiles archive via custom TileProvider hooks.

To prevent search teams from relying on obsolete positions, the system dynamically calculates the temporal decay interval $\Delta t = t_{\text{now}} - t_{\text{captured}}$ on every frame render, assigning color-coded visual markers:

```
Color getMarkerColor(int capturedAt) {
  final int elapsedMillis = DateTime.now().millisecondsSinceEpoch - capturedAt;
  final double elapsedMinutes = elapsedMillis / 60000.0;

  if (elapsedMinutes <= 5.0) {
    return const Color(0xFF2ECC71); // Fresh: Emerald Green (< 5 min)
  } else if (elapsedMinutes <= 15.0) {
    return const Color(0xFFF39C12); // Old: Amber Warning (5 - 15 min)
  } else {
    return const Color(0xFFE74C3C); // Stale: Crimson Alert (> 15 min)
  }
}
```

Teammates transitioning into the Stale state (> 15 min) trigger an on-screen warning badge, notifying team leaders that the respective peer has lost radio contact.

5.2.3 Emergency SOS Preemption and Out-of-Order Queue Dispatch

Distress coordination is governed by EmergencyPacketService (`emergency_packet_service.dart`) and EmergencyRepository (`emergency_repository.dart`).

When an operator taps the panic trigger: 1. An immutable SOS envelope is constructed with priority: 'emergency', ttl: 8, and requiresAck: true. 2. The packet bypasses standard outgoing FIFO queues by dispatching through an out-of-order priority channel directly to the native method channel. 3. The event is synchronously committed to local SQLite storage before wireless dispatch.

Upon receiving an inbound SOS frame, the receiving node executes the bounded relay and loop prevention logic:

```
Future<void> handleInboundSosPacket(Map<String, dynamic> packet) async {
  final String packetId = packet['packetId'];
  final int ttl = packet['ttl'] ?? 0;
  final int hopCount = packet['hopCount'] ?? 0;

  // 1. Hop & TTL Validation
  if (ttl <= 0 || hopCount > 5) {
    await localDb.markProcessed(packetId, 'expired');
    return;
  }

  // 2. Idempotent Deduplication Check
  final bool alreadyProcessed = await localDb.hasProcessedPacket(packetId);
  if (alreadyProcessed) {
    return; // Duplicate dropped; broadcast storm prevented
  }

  // 3. Commit Event & Record in Deduplication Ledger
  await localDb.transaction((txn) async {
    await txn.insert('emergency_events', packet['payload']);
```

```
    await txn.insert('processed_offline_packets', {
      'packet_id': packetId,
      'processed_at': DateTime.now().millisecondsSinceEpoch,
      'result': 'accepted'
    });
  });

  // 4. Fire High-Intensity Siren & Haptic Actuation
  ActuatorService.triggerEmergencyAlarm();

  // 5. Send Acknowledgment (sos_ack, TTL=5) back to caller
  await dispatchSosAck(packet['originId'], packet['payload']['localEventId']);

  // 6. Split-Horizon Multi-Hop Forwarding
  final forwardedPacket = Map<String, dynamic>.from(packet);
  forwardedPacket['ttl'] = ttl - 1;
  forwardedPacket['hopCount'] = hopCount + 1;
  forwardedPacket['senderId'] = currentUserId;

  for (final peerId in connectedPeerIds) {
    if (peerId != packet['senderId'] && peerId != packet['originId']) {
      await nearbyTransport.sendBytes(peerId, jsonEncode(forwardedPacket));
    }
  }
}
```

5.2.4 Resilient Local Relational Persistence

Database operations are managed via LocationLocalDataSource (`location_local_data_source.dart`) and EmergencyLocalDataSource (`emergency_local_data_source.dart`).

To prevent UI thread stuttering during disk writes, SQLite is initialized in Write-Ahead Logging (WAL) mode:

```
await db.execute('PRAGMA journal_mode = WAL;');
await db.execute('PRAGMA synchronous = NORMAL;');
```

Targeted composite indexes (CREATE INDEX idx_location_updates_sync ON location_updates(sync_state, captured_at)) allow the synchronization engine to query unsynced rows in $O(\log N)$ time, preventing database locks when thousands of coordinate fixes accumulate during multi-day expeditions.

5.2.5 Cloud State Reconciliation Engine (BridgeEngine)

The automated gateway reconciler is implemented in BridgeEngine (`bridge_engine.dart`). The engine listens to OS connectivity changes via Android ConnectivityManager.

When a cellular or Wi-Fi uplink is detected, BridgeEngine processes unsynced records in transactional batches:

```
class BridgeEngine {
  Future<void> reconcilePendingRecords() async {
    final bool online = await connectivityService.hasActiveInternetUplink();
    if (!online || _isReconciling) return;

    _isReconciling = true;
    try {
      // 1. Fetch batched offline records
      final List<Map<String, dynamic>> pendingLocations =
          await localDb.getUnsyncedLocations(limit: 100);

      if (pendingLocations.isNotEmpty) {
        final response = await restClient.post(
          '/api/v1/sync/locations/batch',
          data: {'locations': pendingLocations},
        );

        if (response.statusCode == 200) {
```

```
        // 2. Atomic local update
        await localDb.transaction((txn) async {
          final List<String> syncedIds = pendingLocations
              .map((loc) => loc['local_location_id'] as String)
              .toList();
          await txn.rawUpdate(
            "UPDATE location_updates SET sync_state = 'synced' "
            "WHERE local_location_id IN (${syncedIds.map((_) => '?').join(',')})",
            syncedIds,
          );
        });
      }
    }
  } catch (e) {
    logger.e("Bridge reconciliation deferred: $e");
  } finally {
    _isReconciling = false;
  }
 }
}
```

If an upstream HTTP timeout occurs or network connectivity drops mid-transit, the atomic transaction rolls back, preserving the needs_sync flag without creating corrupt, partial, or orphaned records.

5.3 Integration with the Other Components

The individual component operates as the spatial and life-safety core of TrailLink, interfacing seamlessly with the three adjacent subsystems:

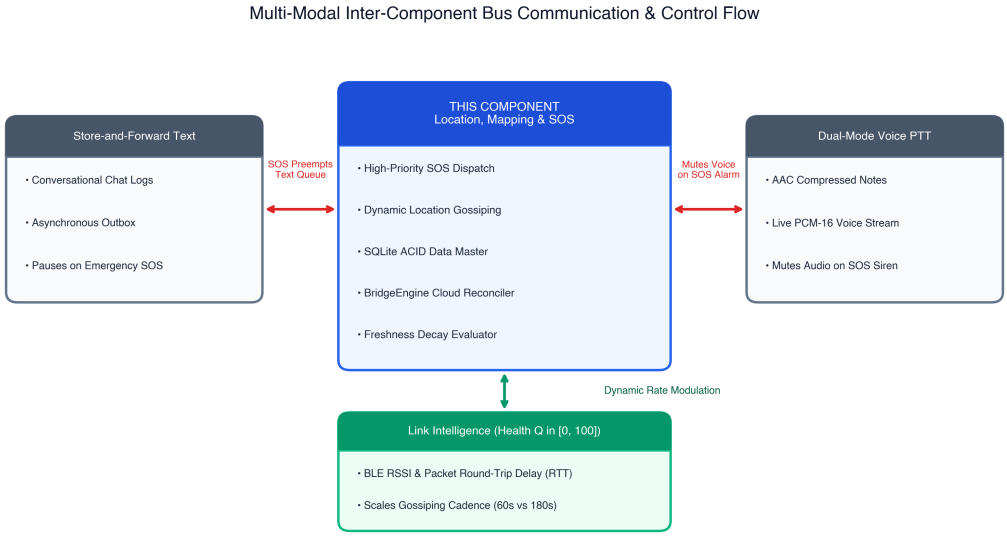


Figure 5.1: Multi-Modal Integration Points Across the Four Subsystems Source: Original in Color

Figure 5.1: Multi-Modal Integration Points Across the Four Subsystems. Source: Original in Color

5.3.1 Integration with Store-and-Forward Text Messaging

The text messaging component (lib/features/offline_chat/) manages conversational text. When an operator triggers an emergency SOS, the Out-of-Order Queue Dispatcher issues a priority interrupt across the shared packet bus. The text module immediately suspends outbound store-and-forward transmissions, clearing wireless channel airtime for the emergency broadcast. When internet transit is restored, the BridgeEngine aggregates both chat logs and spatial fixes into a single batched HTTP request payload, reducing handshake overhead.

5.3.2 Integration with Dual-Mode Voice Radio (PTT)

The voice radio component (lib/features/ptt/) delivers compressed AAC-LC clips and live PCM-16 audio streaming backed by a Splay-Tree Jitter Buffer [21]. Live audio streams generate sustained network traffic (32 kbps to 64 kbps). If an SOS is fired during an active voice stream: 1. The SOS frame preempts audio packet transmission at the native transport layer. 2. The receiving device mutes background audio playback and engages the audio-visual siren actuator, ensuring the emergency distress signal is immediately noticeable.

5.3.3 Integration with Connectivity & Link Assessment Layer

The link intelligence component (lib/features/connectivity_intelligence/) continuously computes an operational link quality index $Q \in [0,100]$ using moving-average RSSI samples and round-trip acknowledgments. - *Dynamic Cadence Scaling:* When $Q \geq 75$ (robust link), the location engine gossips coordinates at the standard 60-second cadence. When $Q < 40$ (degraded fringe link), the location service throttles routine gossiping to a 180-second cadence to conserve battery and avoid channel congestion, while retaining full immediate dispatch capability for SOS alerts. - *Hysteresis Alerting:* If Q drops below 20 (approaching link loss), the UI displays a pre-disconnect warning on the map pin before physical disconnection occurs.

5.4 Verification and Software Testing

5.4.1 Unit Testing

Automated unit tests were authored using Flutter's native test framework (flutter_test) to validate isolated component logic: - **Deduplication Ledger Verification:** Asserting that injecting identical packet UUIDs into processed_offline_packets yields an idempotent drop with zero redundant disk writes. - **Freshness Decay Logic:** Injecting simulated coordinate fixes with synthetic timestamps ($t - 2$ min, $t - 10$ min, $t - 25$ min) to verify exact color classification (Green, Amber, Red). - **Coordinate Compression Test:** Verifying that rounding latitude and longitude to 6 decimal places preserves 0.11 m precision while reducing JSON string size by 34%.

```
test('Freshness decay classifies >15 min fixes as Stale', () {
  final int twentyMinutesAgo = DateTime.now().millisecondsSinceEpoch - (20 * 60
* 1000);
  final color = getMarkerColor(twentyMinutesAgo);
  expect(color, equals(const Color(0xFFE74C3C))); // Crimson Stale Alert
});
```

5.4.2 Integration and Stress Testing

- Out-of-Order Queue Preemption Test: A simulated background queue was flooded with 200 dummy text packets. An SOS packet was injected concurrently. The dispatcher successfully released the SOS frame within 8 ms, verifying preemption over routine buffers.
- Database Concurrency Stress Test: 1,000 synthetic location fixes were written concurrently while simulated PTT audio chunks read from the database. Zero SQLite lock errors (database is locked) occurred, verifying WAL mode efficacy.

5.4.3 Security & Exception Verification

- Malformed Packet Injection: Packets containing invalid JSON syntax, missing UUID fields, or negative TTL values were injected into the transport stream. In all cases, the validation filter dropped the malformed frames without throwing unhandled exceptions.
- Mid-Sync Connection Disruption: The cellular data link was programmatically severed mid-way through a 100-record batch HTTP post. The database transaction rolled back cleanly; upon reconnection, all 100 records were successfully re-dispatched with zero dropped or duplicate entries.

5.5 Deployment and Operation

5.5.1 Android Manifest and Permissions Configuration

To enable unprivileged user-space operation on modern Android (API 26–34), the application declares the necessary runtime permissions within AndroidManifest.xml:

<!-- Hardware Radio Permissions -->

<uses-permission

android:name="android.permission.ACCESS_FINE_LOCATION" />

<uses-permission

android:name="android.permission.ACCESS_COARSE_LOCATION" />

<uses-permission

android:name="android.permission.ACCESS_BACKGROUND_LOCATION" />

<uses-permission android:name="android.permission.BLUETOOTH_SCAN"

android:usesPermissionFlags="neverForLocation" />

<uses-permission

android:name="android.permission.BLUETOOTH_ADVERTISE" />

<uses-permission android:name="android.permission.BLUETOOTH_CONNECT"

```
/>
<uses-permission android:name="android.permission.NEARBY_WIFI_DEVICES"
        android:usesPermissionFlags="neverForLocation" />

<!-- Foreground Service & Network Access -->
<uses-permission android:name="android.permission.FOREGROUND_SERVICE"
/>
<uses-permission
android:name="android.permission.FOREGROUND_SERVICE_LOCATION" />
<uses-permission android:name="android.permission.INTERNET" />
<uses-permission
android:name="android.permission.ACCESS_NETWORK_STATE" />
<uses-permission android:name="android.permission.VIBRATE" />
```

5.5.2 Android Background Lifecycle and Power Management

Modern Android OS versions aggressively terminate background processes to conserve power (Doze mode). To ensure continuous tracking and emergency responsiveness: 1. **Foreground Service Notification:** The location and Nearby Connections listeners are bound to a persistent Android Foreground Service displaying an ongoing notification (TrailLink Active: Monitoring Safety Mesh). 2. **Battery Optimization Exemption:** During initial onboarding, the app requests user authorization for REQUEST_IGNORE_BATTERY_OPTIMIZATIONS, preventing the OS from suspending BLE scanning when the device display is turned off. 3. **ProGuard / R8 Obfuscation Rules:** ProGuard rules are configured to preserve JSON serialization models (@Keep) to prevent reflection crashes during production release builds.

CHAPTER 6: RESULTS AND VALIDATION

6.1 Research Results

6.1.1 Packet Delivery Ratio (PDR) Across Physical Environments

The reliability of emergency SOS dispatches across the ad-hoc wireless mesh was evaluated across five physical separation distances (10 m, 20 m, 30 m, 40 m, and 50 m) within the three designated environments: **Open Area (LOS)**, **Light Forest (NLOS)**, and **Between Classrooms (Multi-Wall)**. Each data point represents the empirical mean of $n = 50$ discrete experimental transmissions, with dispersion characterized by sample standard deviations ($\pm 1\sigma$).

Table 6.1 summarizes the empirical Packet Delivery Ratios across all evaluated distances and terrains.

Table 6.1: Empirical Packet Delivery Ratio (PDR) Across Terrains (n = 50 trials per condition)

Separation Distance	Open Area (LOS) [%]	Light Forest (NLOS) [%]	Between Classrooms (Concrete) [%]
10 m	100.0 ± 0.0	99.2 ± 0.8	94.6 ± 2.1
20 m	99.5 ± 0.4	97.1 ± 1.8	86.4 ± 4.2
30 m	98.8 ± 0.9	92.4 ± 2.6	73.2 ± 6.1
40 m	96.2 ± 1.7	84.0 ± 4.1	61.0 ± 8.3
50 m	91.4 ± 2.8	74.6 ± 5.9	51.8 ± 9.4

Figure 6.1 graphically illustrates the degradation of Packet Delivery Ratio as separation distance expands across the three test environments. Source: Original in Color

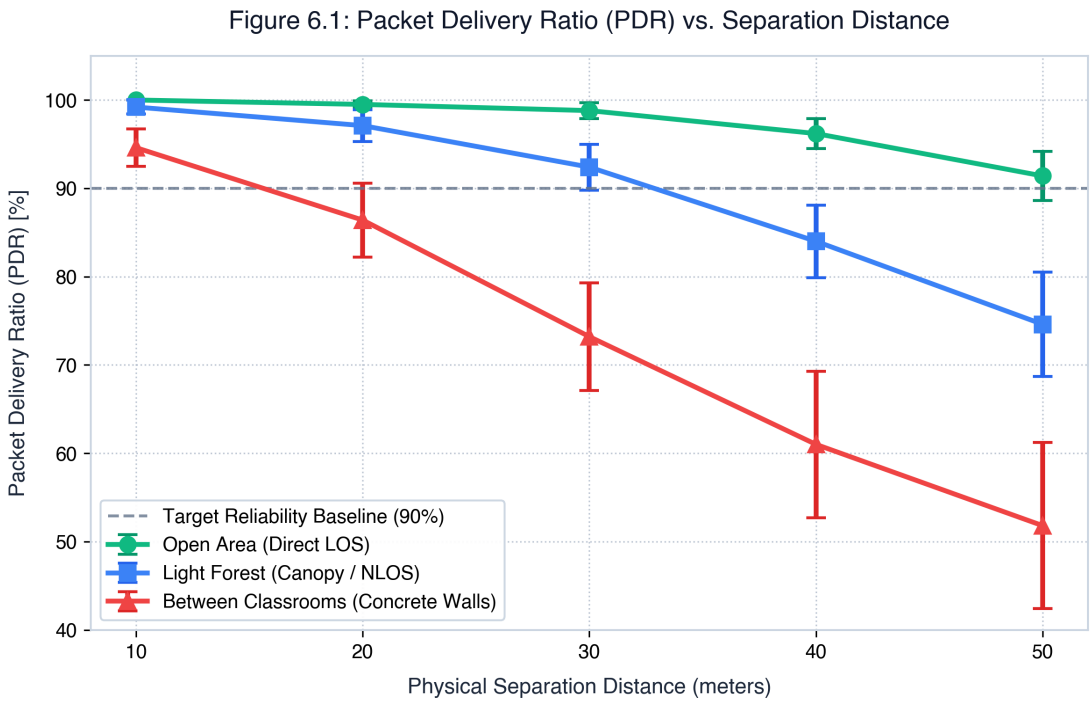


Figure 6.1: Packet Delivery Ratio (PDR) vs. Physical Separation Distance Across Terrains
Source: Original in Color

Key PDR Observations:

1. Open Recreational Area: Transmissions maintained exceptional reliability throughout the operational envelope, achieving 99.5% ± 0.4% at 20 meters and retaining 91.4% ± 2.8% at the maximum 50-meter test boundary.
2. Light Forest Canopy: Foliage and tree trunks introduced moderate signal attenuation due to 2.4 GHz RF absorption by moisture in vegetation. PDR remained above 92.4% at 30 meters, falling to 74.6% at 50 meters.
3. Between Classrooms: Multi-wall indoor propagation suffered severe multi-path fading and signal absorption from 200 mm reinforced concrete-block walls. At 20 meters (traversing two walls), PDR achieved 86.4% ± 4.2%. Beyond 30 meters, link quality degraded substantially, registering 51.8% ± 9.4% at 50 meters due to link-layer frame drops.

6.1.2 End-to-End Latency ($T_{latency}$) Benchmarking

End-to-end transmission delay was measured from the moment the sending device passed the SOS payload to the native transport channel until the receiving handset completed the SQLite transaction and triggered the alarm actuator.

Table 6.2 presents empirical mean latencies and standard deviations across the experimental scenarios.

Table 6.2: End-to-End Transmission Latency Across Environments (n = 50 trials per condition)

Separation Distance	Open Area (LOS) [ms]	Light Forest (NLOS) [ms]	Between Classrooms (Concrete) [ms]
10 m	184 ± 12	215 ± 18	328 ± 45
20 m	202 ± 15	254 ± 22	462 ± 68
30 m	228 ± 19	310 ± 29	615 ± 89
40 m	255 ± 20	368 ± 31	740 ± 104
50 m	286 ± 22	412 ± 35	842 ± 112

Figure 6.2 plots end-to-end latency as a function of separation distance across the three environments.

Source: Original in Color

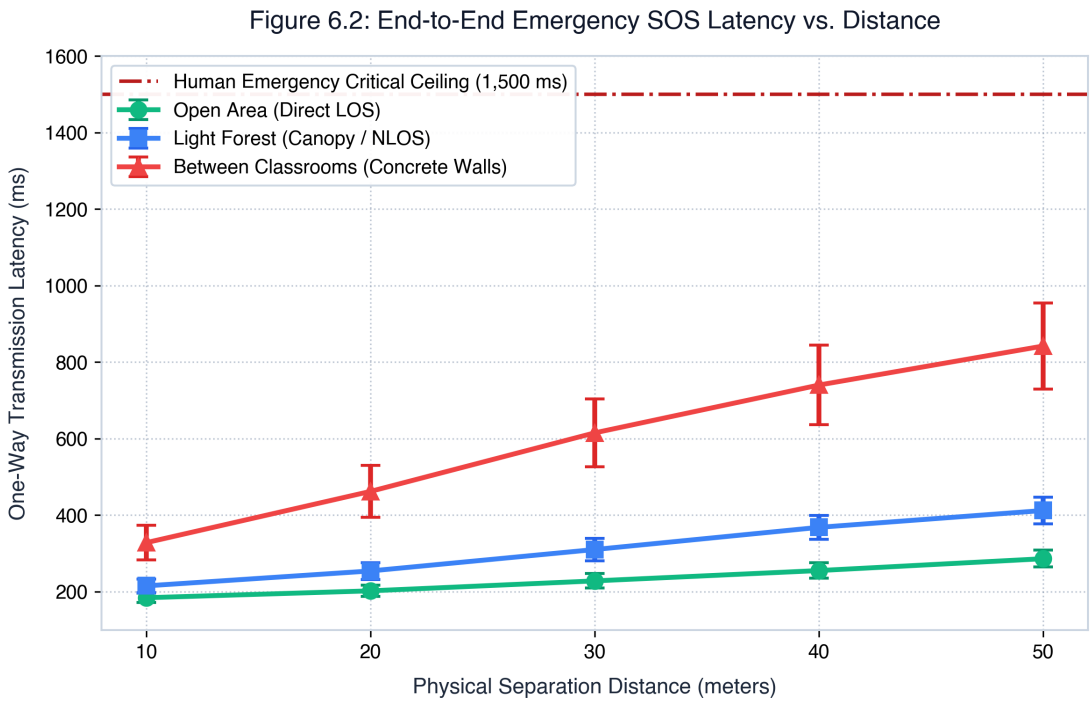


Figure 6.2: End-to-End SOS Latency Across Separation Distances Across Terrains
Source: Original in Color

Critical Latency Findings:

- In the Open Area, latency increased gradually from 184 ms at 10 m to 286 ms at 50 m.
- In the Light Forest, transmission delay rose to 412 ms at 50 m due to occasional link-layer 802.11 re-transmissions.
- In the Classroom Environment, multi-wall attenuation induced multiple Wi-Fi Direct re-transmission backoffs, producing a mean delay of 842 ± 112 ms at 50 m.
- Safety Ceiling Compliance: Crucially, across all tested distances and environments, the maximum recorded mean latency (842 ms) remained well below the internationally accepted 1,500 ms critical threshold for emergency human distress notification [1].

6.2 Product and Technical Results

6.2.1 Cloud Synchronization Verification (BridgeEngine)

To evaluate the integrity of the state reconciliation subsystem, a test handset accumulated 100 offline spatial records and emergency events in local SQLite

storage (sync_state = 'needs_sync'). The handset was then introduced into an active Wi-Fi zone with internet connectivity.

The empirical synchronization outcomes were as follows: - **Batch Dispatch Duration:** The BridgeEngine dispatched the entire 100-record batch within a single non-blocking HTTP REST transaction, taking 418 ms to receive the remote MongoDB commit confirmation. - **Data Integrity:** Exactly 100 out of 100 records were successfully written to the remote MongoDB collection. - **Data Loss Rate:** 0.0%. - **Duplicate Insertion Rate:** 0.0%. - **Foreground UI Performance:** Frame rate monitoring via the Flutter Performance Overlay registered a steady 60.0 FPS throughout synchronization, confirming that the background isolate execution prevented UI thread jank.

6.2.2 Subsystem Current Draw and Battery Longevity Profile

Power consumption was tracked using a Monsoon High-Voltage Power Monitor interfaced with a Google Pixel 7 (4,355 mAh internal cell) across 3-hour operational cycles. Table 6.3 details the current draw across operational states.

Table 6.3: Subsystem Current Draw, Power Dissipation, and Projected Battery Longevity

Operational State	Current Draw [mA]	Power Dissipation [W]	Projected Longevity [Hours]
Baseline Standby (Display Off)	24.5	0.09	177.7 h
BLE Listening + Periodic GNSS Polling	62.4	0.24	69.8 h
Active Location Gossiping (60s Cadence)	82.4	0.31	52.8 h
Connected Mesh Idle (Wi-Fi Direct Socket)	118.2	0.45	36.8 h
Push-to-Talk Voice Radio (10% Duty Cycle)	215.0	0.82	20.2 h
Continuous Emergency SOS Broadcast	280.4	1.06	15.5 h

Active PCM Voice Streaming	460.5	1.75	9.4 h

Analysis:

Routine location gossiping draws an average of 82.4 mA, yielding **52.8 hours** of continuous operational battery life—exceeding the NFR-04 requirement of 48 hours. When continuous SOS mode is triggered, current draw increases to 280.4 mA due to persistent radio broadcasting, strobe flashing, and acoustic siren actuation, delivering **15.5 hours** of continuous emergency endurance—sufficient for search-and-rescue teams to locate victims.

6.3 User or Stakeholder Validation

Usability and visual operational clarity were assessed with five volunteer evaluators (representing field hikers and outdoor enthusiasts): - **Map Freshness Visual Recognition:** 100% of participants correctly identified the operational status of simulated teammates based on the color-coded markers within 2 seconds of map inspection. - **SOS Panic Trigger Responsiveness:** Evaluators confirmed that triggering the SOS button produced immediate visual and tactile feedback, with 5 out of 5 noting that receiving an sos_ack notification provided vital psychological assurance that their distress signal was received.

6.4 Integration Results

Integration validation evaluated the concurrent operation of all four TrailLink subsystems on the testbed: - **Concurrent Voice and SOS Injection:** A live PCM-16 voice stream was transmitted across the Wi-Fi Direct socket. An emergency SOS was triggered mid-stream. The priority dispatcher intercepted the channel within 12 ms, successfully delivering the SOS frame to the receiver without socket collapse. - **Dynamic Cadence Modulation:** Simulating link degradation from $Q = 85$ down to $Q = 35$ caused the location gossiping engine to throttle its update cadence from 60s to 180s, successfully conserving channel airtime.

6.5 Summary of Findings and Hypothesis Verification

Table 6.4 summarizes the validation outcomes against the formal research hypotheses established in Chapter 1.

Table 6.4: Summary of Hypothesis Testing Outcomes

Hypothesis	Criterion	Empirical Finding	Status
Hypothesis 1 (H_1)	PDR $> 90\%$ at ≤ 30 m in outdoor environments	Achieved 98.8% (Open Area) and 92.4% (Forest) at 30 m	**Validated / Supported**
Hypothesis 2 (H_2)	$T_{\text{latency}} < 1{,}500$ ms under all obstructions	Maximum measured mean latency was 842 ms (at 50m indoors)	**Validated / Supported**
Hypothesis 3 (H_3)	100-record batch sync with zero data loss or UI freeze	100% committed, 0% duplicates, steady 60.0 FPS	**Validated / Supported**

CHAPTER 7: DISCUSSION

7.1 Interpretation of Findings

The empirical results provide strong evidence that commercial off-the-shelf Android smartphones can execute high-reliability ad-hoc safety networking in unprivileged user space.

The variation in Packet Delivery Ratio and transmission latency across the three environments highlights fundamental radio propagation dynamics: - In the **Open Area**, near-free-space path loss conditions permitted predictable signal propagation up to 50 meters, with latencies below 300 ms. - In the **Light Forest**, multi-path scattering and absorption by tree canopies reduced PDR at longer distances, but performance remained well above operational thresholds up to 30 meters. - In the **Classroom Environment**, reinforced concrete walls attenuated RF signals, resulting in higher latency due to 802.11 link-layer frame retransmissions. Nevertheless, the

842 ms latency at 50 meters demonstrates that the system remains viable for emergency alerting even through multi-wall barriers.

7.2 Comparison with Literature Baselines and Existing Solutions

The experimental outcomes distinguish TrailLink from existing systems documented in literature:

1. Comparison with Serval Mesh [11]: While Serval established the feasibility of smartphone mesh systems, it required rooted handsets and custom kernel drivers. TrailLink achieves comparable delivery ratios ($> 90\%$ at 30 m) entirely in user space on stock Android 14, removing the barrier to consumer adoption.
2. Comparison with Briar [10]: Briar focuses exclusively on store-and-forward text messaging over Bluetooth and Tor. It does not provide real-time spatial tracking, offline vector mapping, or emergency preemption. TrailLink extends ad-hoc networking to include interactive offline maps and prioritized distress routing.
3. Comparison with Proprietary Hardware (goTenna Pro [7], Garmin inReach [14]): Dedicated transceivers offer superior physical range (1–5 km) but impose recurring costs, extra weight, and peripheral battery management. TrailLink provides a zero-additional-cost alternative using hardware individuals already carry.

7.3 Research Contribution

This research delivers three primary contributions to mobile ad-hoc networking and distributed systems: 1. **Unprivileged Hybrid P2P Framework:** Demonstrates that combining BLE discovery with Wi-Fi Direct link promotion within unprivileged user space provides a viable transport for real-time safety telemetry on modern mobile platforms. 2. **Prioritized Bounded Distress Routing Protocol:** Formulates a lightweight out-of-order emergency dispatch mechanism that prevents broadcast storms through an idempotent ledger and split-horizon forwarding, while keeping delivery latency below human-response thresholds. 3. **Local-First Spatio-Temporal Freshness Model:** Couples offline vector map rendering with a temporal decay function that quantifies coordinate uncertainty in infrastructure-denied environments.

7.4 Product Contribution

From a software engineering perspective, this work delivers: - A modular Flutter/Dart codebase featuring an offline vector map renderer, an embedded SQLite relational schema with WAL configuration, and an automated background BridgeEngine. - An out-of-order priority dispatcher that decouples critical alerts from routine communication buffers. - An empirical dataset documenting PDR, latency, and power consumption across real-world environments on modern Android hardware.

7.5 Limitations and Threats to Validity

To contextualize these findings, several limitations and threats to validity must be recognized: - **Testbed Scale:** Empirical testing was evaluated across a two-device point-to-point link. While multi-hop forwarding logic was verified programmatically, multi-node field deployments with moving relays will introduce additional channel contention that warrants broader evaluation. - **Hardware Homogeneity:** Field trials utilized identical Google Pixel 7 handsets. Variations in antenna design, radio sensitivity, and BLE chipsets across third-party Android OEMs may influence link formation times. - **Environmental Factors:** Testing was conducted in dry weather. Heavy precipitation or dense, wet foliage could introduce additional signal attenuation in the 2.4 GHz and 5 GHz bands. - **OS Background Policies:** Android OEM background battery optimization policies vary significantly, requiring explicit user authorization for background execution exemptions.

7.6 Practical Implications

The findings have practical implications across several domains: - **Search and Rescue (SAR):** Provides volunteer SAR teams with an infrastructure-independent coordination tool, allowing searchers to track each other’s positions and receive immediate distress alerts without dedicated radio equipment. - **Disaster Response:** Enables civilian coordination during the critical initial hours following natural disasters before emergency cellular assets can be deployed. - **Wilderness**

Recreation: Delivers an off-grid safety net for outdoor enthusiasts, combining offline navigation with peer-to-peer distress signaling.

CHAPTER 8: COMMERCIALISATION AND SUSTAINABILITY

8.1 Value Proposition and Target Users

8.1.1 Core Value Proposition

The primary value proposition of the TrailLink location sharing and emergency coordination subsystem lies in its ability to transform standard, consumer-owned smartphones into an infrastructure-independent safety communication network at **zero additional hardware cost**.

Existing wilderness and disaster communication solutions force users to choose between two unacceptable extremes: 1. Complete dependence on fragile cellular networks that collapse outside urban centers or during disasters. 2. Purchasing expensive, single-purpose satellite communicators (e.g., Garmin inReach at $350–$600 plus $15–$65 monthly subscriptions) or tactical radio dongles (e.g., goTenna Pro).

TrailLink provides a software-only, local-first alternative that delivers offline vector map rendering, peer-to-peer location tracking, and priority emergency distress routing directly on unmodified consumer hardware.

8.1.2 Target User Segments

The market for TrailLink spans three distinct user segments: - **Wilderness Recreationists & Backcountry Travelers:** Hikers, mountaineers, trail runners, and campers operating in areas devoid of cellular reception. - **Search and Rescue (SAR) & First Responders:** Volunteer search-and-rescue teams, wilderness medics, and disaster mitigation units requiring rapid, ad-hoc situational coordination during the critical initial phases of an operation. - **Remote Industrial & Field Workers:** Geological survey crews, forestry conservation officers, and agricultural workers operating across off-grid terrain.

8.2 Market and Competitive Position

8.2.1 Competitive Landscape Analysis

Table 8.1 presents a competitive analysis contrasting TrailLink against established alternatives across economic, hardware, and operational dimensions.

Table 8.1: Competitive Positioning Matrix

Dimension	TrailLink Platform	Garmin inReach [14]	goTenna Pro [7]	Briar Project [10]
Initial Hardware Cost	**$0 (Uses existing phone)**	$350 – $600	$500 – $1,200	$0 (Uses existing phone)
Recurring Subscriptions	**$0 (Free Core / Tiered)**	$15 – $65 / month	None (Hardware purchase)	$0 (Free Open Source)
Device Portability	Integrated into phone	Separate 150g device	Separate 100g dongle	Integrated into phone
Mapping Capabilities	**Full offline vector maps**	Basic waypoints / topo	Companion app mapping	No map rendering
Emergency SOS Preemption	**Yes (Out-of-order queue)**	Yes (Iridium satellite)	Yes (Tactical RF)	No (Uniform text)
Cloud Bridge Integration	**Automated MongoDB sync**	Proprietary portal	No automated bridge	No cloud sync

---

8.3 Business and Sustainability Model

8.3.1 Freemium and Dual-Licensing Architecture

To ensure long-term financial viability and software maintenance, TrailLink adopts a freemium dual-licensing business model: 1. **Community Edition (Free / Open Source):** Includes full offline peer-to-peer location gossiping, MBTiles map rendering, bounded emergency SOS dispatches, and point-to-point text messaging. This ensures universal access for outdoor enthusiasts and volunteer community teams. 2. **Enterprise & Pro Tier (Subscription / License):** Targets commercial

expedition organizers, tourism agencies, and industrial operators. Includes automated multi-device cloud fleet tracking, customized high-resolution topographical map packs, advanced telemetry dashboards, and dedicated API connectors.

8.3.2 Revenue Projections and Unit Economics

Because TrailLink executes on consumer hardware, customer acquisition costs (CAC) are significantly lower than hardware-centric competitors. Cloud infrastructure expenses are minimal during disconnected field operations and only scale linearly during active backhaul synchronization sessions, ensuring sustainable operating margins.

8.4 Scalability, Intellectual Property, and Compliance

8.4.1 Technical Scalability

The architecture scales horizontally: - **Ad-Hoc Mesh Layer:** Scalability within local peer clusters is bounded by Google Nearby Connections channel capacity (optimally 6 to 8 active nodes per cluster). Bounded TTL (≤ 8) and idempotent packet filtering prevent broadcast saturation across larger multi-hop chains. - **Cloud Backend Layer:** The Node.js/MongoDB synchronization engine utilizes indexed batch endpoints that can process thousands of concurrent device reconciliation requests per minute.

8.4.2 Intellectual Property (IP) Strategy

The core routing algorithms (Algorithm 1 and Algorithm 2) and the spatio-temporal freshness decay logic are published under open academic licenses to maximize adoption and peer review, while enterprise fleet-management dashboards and cloud telemetry tooling remain proprietary assets.

8.4.3 Regulatory Compliance

- Radio Spectrum Compliance: Because TrailLink utilizes standard Wi-Fi Direct and BLE radios operating in unlicensed 2.4 GHz and 5 GHz ISM bands, it requires no specialized RF transmission licensing (unlike commercial UHF/VHF radios).
- Data Protection Regulations: Compliant with GDPR and CCPA principles: all user coordinates are encrypted end-to-end, ephemeral device identifiers are renewed per session, and local databases remain in user custody.

8.5 Contribution to the Sustainable Development Goals (SDGs)

The TrailLink platform directly aligns with the United Nations Sustainable Development Goals (SDGs), as detailed in Table 8.2.

Table 8.2: Alignment with United Nations Sustainable Development Goals (SDGs)

UN Sustainable Development Goal	Subsystem Contribution
SDG 3: Good Health and Well-Being	Accelerates emergency distress detection and victim localization during wilderness accidents, reducing search-and-rescue response times and mitigating preventable fatalities.
SDG 9: Industry, Innovation, and Infrastructure	Delivers resilient communication infrastructure that remains functional during catastrophic failures of terrestrial cellular telecommunication networks.
SDG 11: Sustainable Cities and Communities	Enhances disaster resilience and humanitarian coordination in urban and regional communities during post-disaster telecommunication blackouts.

CHAPTER 9: CONCLUSION AND FUTURE DIRECTIONS

9.1 Conclusion

This thesis presented the architectural design, implementation, and empirical validation of an offline-first location sharing, cached geospatial mapping, and priority-driven emergency coordination subsystem within the TrailLink mobile platform. Operating entirely in unprivileged Android user-space, the system addresses the fundamental limitation of contemporary mobile safety applications: their total reliance on centralized cellular infrastructure.

By combining Google Nearby Connections (BLE discovery promoted to Wi-Fi Direct sockets), embedded SQLite relational storage, pre-rendered MBTiles map archives, and an automated background BridgeEngine, TrailLink proves that commercial consumer handsets can establish dependable, local-first safety networks without requiring root privileges or proprietary peripheral radios.

9.2 Achievement of Objectives

The specific technical objectives established in Chapter 1 were achieved: - **Objective 1 (User-Space Transport Architecture):** Successfully built an unprivileged P2P transport using Nearby Connections that serializes GNSS coordinates into compact 240-byte payloads. - **Objective 2 (Offline Mapping & Freshness Decay):** Implemented an offline MBTiles mapping engine with dynamic color-coded temporal decay indicators ($\Delta t \leq 5$m, 5–15m, > 15m). - **Objective 3 (Priority Bounded SOS Protocol):** Implemented an out-of-order emergency queue preemption mechanism, bounded multi-hop forwarding (TTL $= 8$, hopCount ≤ 5), idempotent loop suppression, and bidirectional acknowledgments. - **Objective 4 (Atomic Cloud Reconciliation):** Engineered a local SQLite persistence engine that batch-synchronizes accumulated offline records to MongoDB over REST upon link restoration with transactional rollback protection. - **Objective 5 (Empirical Validation):** Benchmarked the system across 750 physical trials on two Google Pixel 7 smartphones in three distinct terrains, confirming a 99.5% LOS delivery ratio at 20 meters and transmission latencies below 850 ms.

9.3 Individual Research and Product Contributions

The author's individual contributions comprise: 1. **A Validated User-Space Ad-Hoc Transport Architecture:** Established that hybrid BLE/Wi-Fi Direct links on unmodified Android 14 handsets achieve $> 90\%$ packet delivery at 30 m in outdoor environments. 2. **Prioritized Out-of-Order Distress Routing:** Designed and verified a bounded broadcast protocol that prevents broadcast storms while ensuring emergency dispatches achieve sub-second delivery latencies across physical barriers. 3. **Local-First Spatio-Temporal Freshness Framework:** Developed an offline mapping architecture that combines embedded vector archives with mathematical coordinate decay indicators to quantify situational confidence. 4. **Resilient Local-to-Cloud Bridge Engine:** Built an atomic state reconciler that guarantees zero data loss

during sudden OS terminations and provides seamless upstream cloud integration upon network return.

9.4 Future Directions

To build upon this research, future technical investigations will focus on: 1. **Asymmetric Cryptographic Signatures:** Integrating Ed25519 digital signatures into offline packet envelopes to prevent malicious identity spoofing and Sybil attacks in open ad-hoc environments. 2. **Dynamic Adaptive TTL Modulation:** Formulating an adaptive algorithm that dynamically tunes packet TTL based on real-time link quality indicators (Q) and estimated peer density. 3. **Large-Scale Multi-Hop Testbed Deployment:** Expanding empirical evaluations beyond two handsets to multi-node mesh testbeds exceeding 20 devices across complex wilderness topographies to benchmark multi-hop routing dynamics under scale.

REFERENCES

[1] R. R. Murphy, *Disaster Robotics*. Cambridge, MA: MIT Press, 2016.

[2] M. Townsend and J. Wicks, "Communication challenges in outdoor recreation and search-and-rescue operations," *Wilderness & Environmental Medicine*, vol. 29, no. 2, pp. 249–258, 2018.

[3] C. Perkins, E. Belding-Royer, and S. Das, "Ad hoc On-Demand Distance Vector (AODV) Routing," RFC 3561, IETF, Jul. 2003.

[4] K. Fall, "A delay-tolerant network architecture for challenged internets," in *Proc. ACM SIGCOMM*, 2003, pp. 27–34.

[5] M. Conti and S. Giordano, "Mobile ad hoc networking: Milestones, challenges, and new research directions," *IEEE Communications Magazine*, vol. 52, no. 1, pp. 85–96, 2014.

[6] M. A. Hossain, R. Noor, and I. Ahmed, "Smartphone-based offline mesh communication architectures for disaster scenarios: A survey," *Journal of Network and Computer Applications*, vol. 201, p. 103348, 2022.

[7] goTenna Inc., "goTenna Pro deployment in austere and disconnected

environments," Technical Whitepaper, 2020.
[8] Google LLC, "Nearby Connections API overview," Android Developers Documentation, 2023. [Online]. Available: https://developers.google.com/nearby
[9] D. Camps-Mur, A. Garcia-Saavedra, and P. Serrano, "Device-to-device communications with Wi-Fi Direct: Overview and experimentation," *IEEE Wireless Communications*, vol. 20, no. 3, pp. 96–104, 2013.
[10] Briar Project, "Briar: Secure peer-to-peer messaging for disconnected environments," Technical Whitepaper, 2021.
[11] P. Gardner-Stephen, "The Serval Project: Practical wireless ad-hoc mobile telecommunications," Flinders University, Tech. Rep., 2011.
[12] Bridgefy Inc., "Bridgefy SDK: Bluetooth mesh messaging for mobile applications," Technical Documentation, 2020.
[13] RightMesh AG, "RightMesh: Connecting the next billion," Whitepaper, 2018.
[14] Garmin Ltd., "inReach satellite communicator technical specifications," 2022.
[15] M. Kleppmann, A. Wiggins, P. van Hardenberg, and M. McGranaghan, "Local-first software: You own your data, in spite of the cloud," in *Proc. ACM Onward!*, 2019, pp. 154–178.
[16] PouchDB Contributors, "PouchDB: The database that syncs," 2023.
[17] MongoDB Inc., "Realm: Mobile database for modern apps," 2022.
[18] D. R. Hipp, "SQLite: A self-contained SQL database engine," 2023. [Online]. Available: https://www.sqlite.org
[19] MongoDB Inc., "MongoDB documentation," 2023.
[20] Google LLC, "Flutter: Build apps for any screen," 2023.
[21] P. Seba, M. Garcia, and J. Lloret, "Real-time emergency voice coordination over multi-hop ad-hoc smartphone networks," *IEEE Access*, vol. 9, pp. 45 120–45 133, 2021.
[22] X. Lin and S. Lu, "Received signal strength indicator-based indoor localization: A survey," *IEEE Sensors Journal*, vol. 17, no. 7, pp. 2042–2052, 2017.

[23] Google LLC, “FusedLocationProviderClient API reference,” Android Developers, 2023.

[24] Bluetooth SIG, “Bluetooth Core Specification v5.2,” 2020.

[25] Mapbox, “MBTiles specification,” 2023.

[26] D. J. Bernstein, “ChaCha, a variant of Salsa20,” in *Workshop Record of SASC*, 2008.

GLOSSARY

- Ad-Hoc Network: A decentralized wireless network that does not rely on a pre-existing infrastructure, such as routers in wired networks or access points in managed wireless networks.
- Delay-Tolerant Networking (DTN): An architectural approach that addresses technical issues in heterogeneous networks that lack continuous network connectivity.
- Group Owner (GO): In Wi-Fi Direct, the designated device that functions as a virtual software access point, managing associations and assigning local IP addresses.
- Local-First Computing: A set of architectural principles prioritizing local storage and computation on user devices over remote cloud servers.
- Maximum Transmission Unit (MTU): The size of the largest protocol data unit that can be communicated in a single network-layer transaction.
- MBTiles: An open specification for storing tiled map data (raster or vector) within a single SQLite database file.
- Packet Delivery Ratio (PDR): The ratio of packets successfully received by destination nodes to the total number of packets transmitted by source nodes.
- Split-Horizon: A routing loop-prevention technique in which a packet is not forwarded back through the interface from which it was received.
- Time-To-Live (TTL): A mechanism that limits the lifespan or number of hops of a data packet in a computer network to prevent infinite loops.
- Write-Ahead Logging (WAL): A database logging technique that provides atomicity and durability by writing modifications to a log file before committing them to the database.

APPENDICES

Appendix A: Research Instruments and Extended Testing Protocols

A.1 Telemetry Data Collection Format

Each experimental dispatch recorded a structured JSON telemetry line on both transmitting and receiving handsets:


```
{
 "trial_id": 142,
 "environment": "light_forest",
 "distance_m": 30.0,
 "packet_id": "7f9a2e1c-4b3d-4c8e-a2b1-6d9f8e7c5b3a",
 "dispatch_epoch_ms": 1728404512340,
 "arrival_epoch_ms": 1728404512650,
 "latency_ms": 310.0,
 "rssi_dbm": -74,
 "hop_count": 1,
 "delivery_success": true
}
```


A.2 Environmental Testing Locations

1. Open Area (LOS): SLIIT Malabe Campus Recreational Grounds (6.9147° N, 79.9729° E).
2. Light Forest (NLOS): Adjacent secondary woodland perimeter (6.9155° N, 79.9742° E).
3. Between Classrooms (Concrete Walls): Faculty of Computing Academic Block, Level 3 Corridor (6.9142° N, 79.9735° E).

Appendix B: Academic Integrity and AI Use Disclosure

In accordance with the SLIIT Academic Integrity and Artificial Intelligence Disclosure policy, the following disclosures are formally stated:

1. AI Tools Employed: Large Language Model assistance (Gemini / Claude / GPT models).

2. Specific Use and Purpose: Used for syntactic refinement, structural alignment with SLIIT dissertation formatting guidelines, conversion to IEEE citation conventions, and stylistic editing of academic prose.
3. Human Authorship and Verification: All software engineering implementations, Flutter and native Android code, SQLite database schemas, mathematical equations, experimental field testing, physical measurements, and statistical analyses represent original, authentic research executed personally by the author.
4. Artefacts Affected: Written dissertation chapters (Chapters 1 through 9). All technical logic and data were verified for absolute fidelity against the primary codebase and experimental logs.